



# Modelagem de Sistemas

2026/2



Profa. Dra. Ana Márcia Debiasi Duarte  
Curso de Sistemas de Informação  
**UNOESC - Universidade do Oeste de Santa Catarina**



Esta apostila tem como objetivo desenvolver a capacidade de analisar, abstrair e representar sistemas de informação por meio de modelos conceituais e técnicos, utilizando princípios de Engenharia de Software, Orientação a Objetos, UML e ferramentas de Inteligência Artificial como apoio ao processo de modelagem. Ao final da disciplina, o estudante será capaz de compreender problemas de negócio, construir modelos consistentes, validar soluções e utilizar a IA de forma crítica e responsável para aumentar a produtividade e a qualidade dos artefatos produzidos.

Caro estudante,

Bem-vindo ao curso do Componente **Modelagem de Sistemas** que tratará os conceitos teóricos para aprofundar seus conhecimentos sobre como modelar um sistema de software. Além de capacitar na prática como usar as técnicas e ferramentas para elaborar uma especificação que possa ser implementada como um software funcional e de qualidade.

Toda a nossa rotina de aulas está descrita no Plano de Ensino do Componente que você tem acesso no Sistema Acadêmico.

Esperamos que, até o final deste Componente você possa:

- Compreender os conceitos de Modelagem de Software;
- Entender e ser capaz de fazer modelagens usando os conceitos de Orientação à Objetos;
- Elaborar Diagramas e textos de especificação nos padrões consolidados de modelagem;

Para tanto, teremos aulas organizadas semanalmente e o conteúdo distribuído em nosso calendário.

O conteúdo base é organizando aqui nesta apostila. Conteúdos complementares ou instruções específicas estarão disponíveis no ambiente do Componente no Sistema Acadêmico, onde todos os estudantes têm acesso.

Vamos, então, iniciar nossas aulas?

Bons estudos!

*1ª Ed., Junho de 2026.*

*Todos os direitos são reservados a autora. Reprodução total ou parcial proibida.*

**I****Unidade 1 - Princípios da Modelagem****1. Princípios da Modelagem**

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 1.1   | Modelagem de Sistemas e Engenharia de Software          | 7  |
| 1.1.1 | Introdução                                              | 7  |
| 1.1.2 | Conteúdo Técnico: Fundamentos da Modelagem              | 7  |
| 1.1.3 | Outros Temas Importantes                                | 9  |
| 1.1.4 | O Olhar do Engenheiro de Software                       | 12 |
| 1.1.5 | Estudo de Caso Integrador: Clínica de Saúde (Etapa 1)   | 12 |
| 1.1.6 | Exercícios de Fixação                                   | 13 |
| 1.2   | Modelagem de Domínio e Engenharia de Requisitos         | 16 |
| 1.2.1 | Introdução                                              | 16 |
| 1.2.2 | Conteúdo Técnico: Domínio e Requisitos                  | 16 |
| 1.2.3 | O olhar do Engenheiro de Software                       | 25 |
| 1.2.4 | Estudo de Caso Integrador: Clínica de Saúde (Etapa 1.2) | 25 |
| 1.2.5 | Exercícios de Fixação                                   | 25 |
| 1.3   | Considerações Finais                                    | 27 |

**II****Unidade 2 - Conceitos de Orientação a Objetos****2. Orientação a Objetos e Descoberta de Classes**

|       |                                                                              |    |
|-------|------------------------------------------------------------------------------|----|
| 2.1   | Introdução à Unidade                                                         | 30 |
| 2.2   | Fundamentos de Objetos e Descoberta de Classes                               | 31 |
| 2.2.1 | Classes vs. Objetos: O Molde e a Instância                                   | 31 |
| 2.2.2 | Atributos e Responsabilidades (Saber vs. Fazer)                              | 33 |
| 2.2.3 | Os Pilares da OO Aplicados à Modelagem                                       | 34 |
| 2.3   | Diagramas da UML                                                             | 36 |
| 2.4   | Diagrama de Classes                                                          | 37 |
| 2.5   | Como Descobrir Classes Conceituais: O Método Larman                          | 40 |
| 2.6   | O Olhar do Engenheiro de Software: Hiato de Representação e Modelos Anêmicos | 43 |
| 2.7   | Estudo de Caso Integrador: Clínica de Saúde (Etapa 2)                        | 43 |
| 2.8   | Guia Prático: Engenharia de Prompts para Descoberta de Classes               | 43 |

|       |                                             |    |
|-------|---------------------------------------------|----|
| 2.8.1 | Princípios da Modelagem Assistida por IA    | 44 |
| 2.8.2 | Templates de Prompts Estruturados           | 44 |
| 2.8.3 | Roteiro Prático de Trabalho para as Equipes | 47 |
| 2.9   | Exercícios de Fixação                       | 48 |

### III

## Unidade 3 - Padrões e Uso dos Diagramas

### 3. Padrões e Uso dos Diagramas

### IV

## Unidade 4 - Guia Prático: Engenharia de Prompts

### 4. Guia Prático: Engenharia de Prompts

|       |                                             |    |
|-------|---------------------------------------------|----|
| 4.1   | Prompts para Descoberta de Classes          | 59 |
| 4.1.1 | Princípios da Modelagem Assistida por IA    | 59 |
| 4.1.2 | Templates de Prompts Estruturados           | 60 |
| 4.1.3 | Roteiro Prático de Trabalho para as Equipes | 62 |





# Unidade 1 - Princípios da Modelagem

# 1. Princípios da Modelagem

## Conteúdo da Unidade

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 1.1   | Modelagem de Sistemas e Engenharia de Software          | 7  |
| 1.1.1 | Introdução                                              | 7  |
| 1.1.2 | Conteúdo Técnico: Fundamentos da Modelagem              | 7  |
| 1.1.3 | Outros Temas Importantes                                | 9  |
| 1.1.4 | O Olhar do Engenheiro de Software                       | 12 |
| 1.1.5 | Estudo de Caso Integrador: Clínica de Saúde (Etapa 1)   | 12 |
| 1.1.6 | Exercícios de Fixação                                   | 13 |
| 1.2   | Modelagem de Domínio e Engenharia de Requisitos         | 16 |
| 1.2.1 | Introdução                                              | 16 |
| 1.2.2 | Conteúdo Técnico: Domínio e Requisitos                  | 16 |
| 1.2.3 | O olhar do Engenheiro de Software                       | 25 |
| 1.2.4 | Estudo de Caso Integrador: Clínica de Saúde (Etapa 1.2) | 25 |
| 1.2.5 | Exercícios de Fixação                                   | 25 |
| 1.3   | Considerações Finais                                    | 27 |

## 1.1 Modelagem de Sistemas e Engenharia de Software



**Objetivo:** Desenvolver a compreensão sobre o papel da modelagem como uma atividade central da Engenharia de Software, capacitando o acadêmico a utilizar a **abstração** para reduzir a complexidade e a distinguir as etapas de **análise** e **projeto** na construção de sistemas

### 1.1.1 Introdução

**Apresentação da Unidade:** O desenvolvimento de software profissional não se resume à escrita de código. É um processo intelectual de compreensão de problemas do mundo real e sua tradução em soluções tecnológicas. A modelagem é a linguagem que permite essa tradução, servindo como uma planta baixa para o engenheiro de software.

**Ao concluir esta unidade, você será capaz de:**

- Compreender a importância estratégica da modelagem no ciclo de vida de desenvolvimento;
- Explicar o papel da abstração na redução da complexidade;
- Diferenciar as atividades de análise (foco no problema) e projeto (foco na solução).

**Por que este conteúdo é importante?** A maior parte das falhas em projetos de software ocorre por má compreensão dos requisitos, e não por falhas na programação. Dominar a modelagem permite "fazer a coisa certa" antes de se preocupar em "fazer a coisa da maneira certa".

### 1.1.2 Conteúdo Técnico: Fundamentos da Modelagem

#### O que é Modelagem?

Modelar é o ato de construir uma representação simplificada da realidade. Um modelo não é uma cópia idêntica do mundo real, mas sim uma representação que enfatiza intencionalmente certos aspectos de um fenômeno enquanto ignora outros que não são relevantes para o problema em questão.

**Exemplo prático:** Considere um mapa. Um completo de Santa Catarina enfatiza todos os aspectos como divisão por regiões, municípios, rodovias e rios. Já um mapa dos rios de Santa Catarina ignora aspectos do estado e traz todas as informações sobre os rios.

Ambos são modelos da realidade, como é possível observar na Figura 1.1, mas com propósitos diferentes.

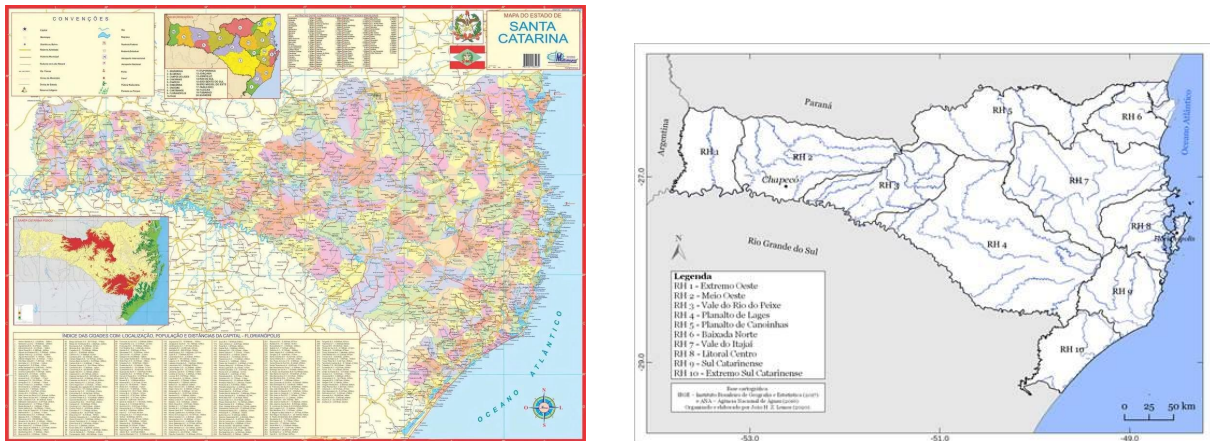


Figura 1.1: Mapas de SC - Diferenças entre propósitos da modelagem



Segundo Eric Evans, um modelo de domínio não é um diagrama específico, mas a ideia organizada que o diagrama pretende transmitir — uma abstração seletiva do conhecimento

### Abstração: A ferramenta contra a Complexidade

A **abstração** é o ato de concentrar-se nas qualidades essenciais ou gerais de coisas semelhantes, omitindo detalhes desnecessários. É o principal mecanismo humano para lidar com a complexidade.

- **Redução da Complexidade:** Ao utilizar abstrações, removemos a "complexidade acidental" (detalhes de implementação) para focar na "complexidade essencial" do problema de negócio.
- **Precisão:** O propósito da abstração não é ser vago, mas criar um novo nível semântico onde o engenheiro possa ser absolutamente preciso sobre o que o sistema deve fazer.

### Exemplo de Níveis de Abstração:

1. Nível Alto: A abstração *Processar Pagamento* foca no negócio (quem paga, quanto, quando).
2. Nível Baixo: A implementação técnica foca no código (quais classes, conexões com banco de dados, protocolos de rede).

### Atividades de Análise versus Projeto (Design)

Dentro da Engenharia de Software, o processo de modelagem é dividido em duas grandes perspectivas: análise e projeto. Veja na Tabela 1.1 as diferenças entre elas.

Tabela 1.1: Diferença entre análise e projeto

| Atividade               | Foco Principal                  | Pergunta Chave                       | Objetivo                                                  |
|-------------------------|---------------------------------|--------------------------------------|-----------------------------------------------------------|
| <b>Análise</b>          | Investigação do <b>Problema</b> | <b>O QUE</b> o sistema deve fazer?   | “Fazer a coisa certa” (Análise de Requisitos).            |
| <b>Projeto (Design)</b> | Definição da <b>Solução</b>     | <b>COMO</b> o sistema vai funcionar? | “Fazer a coisa da maneira certa” (Arquitetura e Objetos). |

**Definições Detalhadas:**

- **Análise:** Investiga o domínio do problema. Resulta em modelos que descrevem características estáticas e dinâmicas do negócio. Exemplo: Identificar que um avião precisa de um número de série e um piloto para realizar um voo.
- **Projeto (Design):** Processo que utiliza os produtos da análise para produzir uma especificação técnica para a implementação. Exemplo: Definir que a classe *Aviao* terá um atributo privado *tailNumber* e um método *getFlightHistory()*.

### 1.1.3 Outros Temas Importantes

**Diferentes ambientes organizacionais de desenvolvimento de software**

Vamos falar dos ambientes que podemos encontrar para trabalhar com desenvolvimento de software. Aqui vamos fazer um resumo e uma simplificação dos diferentes ambientes que podemos encontrar para atuar como Engenheiros de Software.

Embora os princípios da Engenharia de Software sejam os mesmos, a forma como os sistemas são desenvolvidos varia significativamente conforme o contexto organizacional. De maneira geral, podem ser identificados três ambientes bastante comuns de desenvolvimento de software, cada um com características, desafios e processos próprios.

**1. Empresas de desenvolvimento de software sob demanda**

Neste modelo, a **a empresa de software desenvolve sistemas específicos para um cliente**. Cada projeto possui objetivos, requisitos, prazos e restrições próprios, sendo tratado como um produto único.

O processo de desenvolvimento normalmente inicia com o levantamento detalhado dos requisitos junto ao cliente, seguido pelas etapas de análise, modelagem, implementação, testes e implantação. Como o software é criado para atender às necessidades particulares de uma organização, a comunicação com o

cliente é intensa durante todo o projeto.

Principais características

- cada projeto é único;
- forte interação com o cliente;
- requisitos podem sofrer mudanças durante o desenvolvimento;
- sucesso depende do correto entendimento das necessidades do cliente;
- equipes trabalham simultaneamente em diferentes projetos.

Principais desafios

- compreender corretamente os requisitos do cliente;
- controlar mudanças de escopo;
- cumprir prazos e orçamento;
- equilibrar qualidade, custo e tempo.

## **2. Empresas desenvolvedoras de produtos de software**

Nesse ambiente, a **empresa desenvolve um produto destinado a diversos clientes**. Em vez de produzir um sistema exclusivo para cada organização, mantém um único produto (ou uma família de produtos) que pode ser configurado ou customizado conforme as necessidades de cada cliente.

São exemplos desse modelo sistemas ERP, softwares de gestão empresarial, sistemas hospitalares, plataformas educacionais, sistemas de automação comercial e aplicativos comercializados em larga escala.

Nesse contexto, uma alteração realizada no produto pode impactar centenas ou milhares de clientes. Por isso, existe uma preocupação constante com compatibilidade, versionamento, manutenção, evolução e qualidade das novas funcionalidades.

Principais características

- um mesmo produto atende vários clientes;
- existe evolução contínua do software;
- novas versões são disponibilizadas periodicamente;
- parte das funcionalidades pode ser configurável ou customizável;
- manutenção corretiva e evolutiva ocorre continuamente.

Principais desafios

- manter compatibilidade entre versões;
- atender necessidades comuns de diferentes clientes;

- evitar que uma alteração gere problemas para clientes já atendidos;
- administrar diferentes configurações do mesmo produto;
- equilibrar inovação com estabilidade.

### **3. Áreas de Tecnologia da Informação de organizações**

Muitas empresas têm como atividade principal negócios que não envolvem o desenvolvimento de software, como indústrias, bancos, hospitais, universidades, cooperativas, supermercados ou órgãos públicos. Nesses casos, existe uma área de Tecnologia da Informação responsável por desenvolver, integrar ou manter sistemas que apoiam as operações da própria organização.

O objetivo não é comercializar software, mas utilizar sistemas de informação para tornar os processos internos mais eficientes, apoiar a tomada de decisão e aumentar a competitividade da empresa.

Os profissionais de TI trabalham próximos às áreas de negócio, buscando compreender suas necessidades e transformá-las em soluções tecnológicas.

Principais características

- o software apoia as atividades da própria organização;
- os usuários pertencem à empresa;
- existe grande integração entre processos de negócio e sistemas de informação;
- frequentemente há integração entre sistemas desenvolvidos internamente e softwares adquiridos de terceiros.

Principais desafios

- compreender os processos de negócio;
- priorizar demandas de diferentes departamentos;
- integrar sistemas distintos;
- garantir disponibilidade e segurança das informações;
- atender rapidamente às mudanças organizacionais.

Agora vamos olhar para a Tabela 1.2 a comparação entre esses três tipos de ambientes e vamos refletir como isso impacta o nosso trabalho como engenheiros de software realizando a atividade de modelagem de sistemas.

Tabela 1.2: Comparação entre os ambientes de desenvolvimento de software

| Aspecto                  | Software sob demanda            | Produto de software                                     | Área de TI                                  |
|--------------------------|---------------------------------|---------------------------------------------------------|---------------------------------------------|
| <b>Cliente</b>           | Um cliente específico           | Diversos clientes                                       | A própria organização                       |
| <b>Objetivo</b>          | Resolver um problema específico | Comercializar um produto reutilizável                   | Apoiar os processos internos                |
| <b>Requisitos</b>        | Exclusivos do cliente           | Comuns a vários clientes, com possíveis customizações   | Definidos pelas áreas de negócio            |
| <b>Evolução</b>          | Conforme contrato do projeto    | Contínua, com novas versões                             | Conforme necessidades organizacionais       |
| <b>Principal desafio</b> | Entender corretamente o cliente | Evoluir o produto sem prejudicar os clientes existentes | Alinhar tecnologia aos objetivos do negócio |

Independentemente do ambiente, a modelagem de sistemas desempenha um papel fundamental para compreender o problema, documentar requisitos, comunicar soluções entre as partes interessadas e reduzir riscos durante o desenvolvimento do software. Entretanto, a forma como os modelos são utilizados e o foco de cada projeto variam conforme o contexto organizacional em que o desenvolvimento ocorre.

#### 1.1.4 O Olhar do Engenheiro de Software

**Investigação antes da Construção:** Um erro clássico é começar a desenhar diagramas UML ou escrever classes antes de entender o domínio. O olhar do engenheiro deve ser o de um investigador: ele deve ouvir, observar e questionar os especialistas do negócio para identificar ambiguidades antes que elas se tornem defeitos (*bugs*) caros no código. **A modelagem é, antes de tudo, uma atividade intelectual de compartilhamento de conhecimento.**

#### 1.1.5 Estudo de Caso Integrador: Clínica de Saúde (Etapa 1)

**Cenário:** Uma clínica de nutrição e massoterapia quer automatizar sua agenda.

**Tarefa de Análise:** Identifique os elementos essenciais do negócio ignorando a tecnologia.

- **Atores:** Quem são as pessoas envolvidas? (Paciente, Nutricionista).
- **Entidades:** O que precisa ser registrado? (Consulta, Plano de Tratamento).



- **Abstração:** Note que, para a clínica, a "cor dos olhos" do paciente é um detalhe irrelevante (pode ser omitido), mas seu "histórico de alergias" é essencial.

**IA como Parceira:** Peça à IA: "Aja como analista. Para uma clínica de bem-estar, descreva o processo de 'Agendamento de Consulta'. Quais informações são indispensáveis para garantir que não haja choque de horários?"

### 1.1.6 Exercícios de Fixação



Para realizar em papel e caneta, sem consulta, para treinar para a prova

1. **Conceitual 1.1:** Defina o conceito de modelo segundo a Engenharia de Software e explique a diferença entre um modelo e uma cópia fiel da realidade.
2. **Conceitual 1.2:** Martin Fowler define três modos distintos de aplicação da linguagem UML: Esboço (*Sketch*), Planta (*Blueprint*) e Linguagem de Programação. Explique o propósito da **UML como Esboço** e em que momento ela é mais utilizada.
3. **Abstração 2.1:** Como a abstração auxilia o analista de sistemas a lidar com a complexidade de um domínio de negócio desconhecido?
4. **Abstração 2.2:** Explique o conceito de "Níveis de Abstração" e forneça um exemplo de uma abstração de nível alto e uma de nível baixo para o processo de "Agendamento" na Clínica de Bem-estar.
5. **Comparativo 3.1:** Apresente três diferenças fundamentais entre as atividades de Análise e Projeto (Design).
6. **Comparativo 3.2:** Diferencie os ambientes de "Desenvolvimento sob Demanda" (*Bespoke*) e "Produtos de Software" (*Generic Products*) quanto à origem dos requisitos e à evolução do sistema.
7. **Cenário 4.1:** Um cliente solicita um "Sistema de Vendas". Liste 3 atividades que pertencem à fase de Análise e 3 que pertencem à fase de Projeto para este pedido.
8. **Cenário 4.2:** No contexto da Etapa 1 do Estudo de Caso (Clínica), identifique uma informação do paciente que seria considerada "essencial" pela abstração e uma que seria "irrelevante". Justifique com base no domínio da clínica.
9. **Reflexão 5.1:** Explique a frase de Larman: "Análise é fazer a coisa certa; Projeto é fazer a coisa da

maneira certa".

10. **Reflexão 5.2:** Por que se afirma que o olhar do engenheiro de software deve ser o de um "investigador" antes de iniciar a construção dos modelos visuais?
11. **Questão ENADE 1.1:** A Engenharia de Software moderna encara a modelagem não apenas como a criação de diagramas, mas como a construção de uma representação simplificada da realidade, preservando elementos relevantes para compreender um problema e discutir soluções. Segundo Eric Evans, um modelo de domínio é uma abstração rigorosamente organizada do conhecimento. Nesse contexto, a abstração atua como a principal ferramenta para lidar com a complexidade, permitindo ao engenheiro de software navegar entre diferentes níveis semânticos.

A respeito dos conceitos de modelagem, abstração e as atividades de análise e projeto, avalie as afirmações a seguir.

- I. Um modelo eficaz deve buscar ser uma cópia fiel da realidade, incluindo o máximo de detalhes do mundo real para garantir que o código-fonte final seja preciso e livre de ambiguidades.
- II. A abstração permite ao analista focar na "complexidade essencial" do problema de negócio, omitindo detalhes de implementação técnica que caracterizam a "complexidade acidental".
- III. No ciclo de vida de desenvolvimento de sistemas, a atividade de Análise foca na investigação do problema (o que o sistema deve fazer), enquanto a atividade de Projeto (Design) foca na definição da solução conceitual (como o sistema irá funcionar).

É correto o que se afirma em:

- A) I, apenas.
  - B) III, apenas.
  - C) I e II, apenas.
  - D) II e III, apenas.
  - E) I, II e III.
12. **Questão ENADE 1.2:** A Engenharia de Software incorpora cinco atividades de arcabouço fundamentais: comunicação, planejamento, modelagem, construção e implantação. A respeito da atividade de **modelagem**, avalie as afirmações a seguir:
- I. A modelagem tem como objetivo principal criar representações que ajudem a entender o quadro geral e como as partes do sistema se encaixam arquiteturalmente.
  - II. Modelos de análise representam o software do ponto de vista técnico e de implementação, enquanto modelos de projeto focam na visão do cliente.

III. Abstração e refinamento são conceitos complementares: a primeira suprime detalhes de baixo nível, enquanto o segundo ajuda a revelá-los gradualmente.

É correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

## 1.2 Modelagem de Domínio e Engenharia de Requisitos



**Objetivo:** Capacitar o acadêmico a transpor a barreira entre o problema de negócio e a especificação técnica, utilizando a **Modelagem de Domínio** para compreender a lógica organizacional e a **Engenharia de Requisitos** para documentar as capacidades e restrições que o sistema deve atender

### 1.2.1 Introdução

**Apresentação da Unidade:** O sucesso de um software não é medido apenas pela ausência de erros no código, mas pela sua utilidade para o negócio. Nesta unidade, aprenderemos que o software deve ser um reflexo do domínio para o qual foi criado. Investigaremos como a comunicação falha entre técnicos e especialistas de negócio é a raiz da maioria dos problemas em projetos e como a Engenharia de Requisitos fornece o arcabouço para mitigar esses riscos.

**Ao concluir esta unidade, você será capaz de:**

- Definir e identificar o domínio e subdomínios de um sistema;
- Identificar e classificar requisitos funcionais e não funcionais utilizando o modelo FURPS+;
- Compreender a distinção entre requisitos de usuário e de sistema;
- Aplicar o conceito de Linguagem Ubíqua (DDD) para unificar a comunicação da equipe.

**Por que este conteúdo é importante?** Projetos de software frequentemente sofrem do efeito "Telefone Sem Fio": o cliente pede algo, o analista entende outra coisa e o desenvolvedor implementa uma terceira. Dominar essas técnicas garante que a equipe "fale a mesma língua" e construa a solução correta para o problema real.

### 1.2.2 Conteúdo Técnico: Domínio e Requisitos

#### O que é um Domínio?

O **domínio** é a área de conhecimento ou atividade específica em torno da qual o software é construído. Por exemplo, em um sistema bancário, o domínio envolve contas, transferências e juros.

- **Especialistas de Domínio:** São as pessoas que entendem profundamente o negócio (ex: o nutricionista na clínica), mas que não necessariamente entendem de software.

## A Linguagem Ubíqua (Domain-Driven Design - DDD)

Para evitar falhas de comunicação, Eric Evans propõe a **Linguagem Ubíqua** (Ubiquitous Language): uma linguagem única, rigorosa e compartilhada por todos os envolvidos (desenvolvedores e especialistas de negócio).

- **Regra de Ouro:** Se um termo mudar no negócio, ele deve mudar na documentação, nos diagramas e, obrigatoriamente, no código-fonte.
- **Exemplo:** Se o dono da clínica chama os clientes de "Pacientes", o código não deve usar a classe `Client`, mas sim `Paciente`.

A Figura 1.2 mostra como a linguagem ubíqua deve fazer a comunicação entre os técnicos e os analistas de negócio.

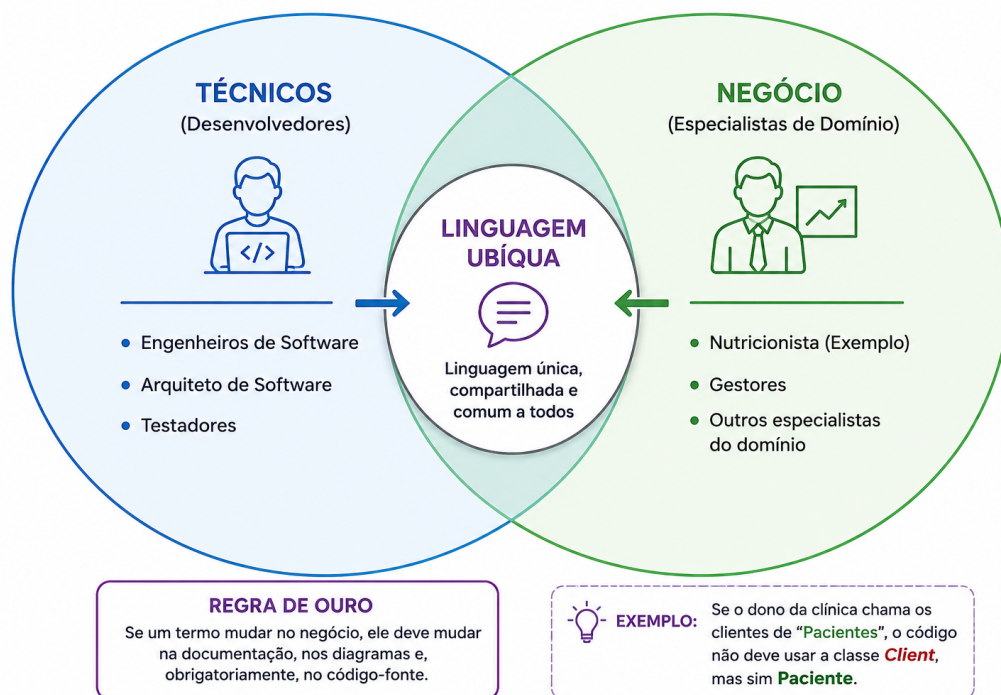


Figura 1.2: Linguagem Ubíqua (Ubiquitous Language)

## Engenharia de Requisitos (ER)

Requisitos são as **capacidades** (o que deve fazer) e as **condições** (restrições) às quais o sistema deve se conformar.

A Engenharia de Requisitos não é meramente uma fase inicial do projeto, mas um processo disciplinado que atua como a ponte fundamental entre as necessidades nebulosas dos stakeholders (ver a definição a seguir no texto) e a precisão exigida pela construção de software.

Sommerville a define como o processo de descobrir, analisar, documentar e verificar os serviços fornecidos pelo sistema e suas restrições operacionais.

Para Pressman, este processo é composto por sete tarefas cruciais que frequentemente ocorrem em paralelo: **concepção** (definição do escopo), **elicitação** (extração de necessidades), **elaboração** (refinamento do modelo), **negociação** (priorização de conflitos), **especificação** (documentação), **validação** (checagem de qualidade) e **gestão** (controle de mudanças). O grande desafio da ER é que, em muitos casos, os clientes não sabem exatamente o que querem, ou suas necessidades mudam à medida que o sistema evolui. A 1.3 mostra as etapas da Engenharia de Requisitos e também as suas relações.



Figura 1.3: Processo da Engenharia de Requisitos

**Exemplo Real:** Imagine o desenvolvimento de um sistema de tráfego aéreo. Se a Engenharia de Requisitos falhar em captar a restrição de que o tempo de resposta deve ser inferior a milissegundos, o software pode estar funcionalmente correto (ele mostra os aviões), mas ser catastroficamente perigoso na prática.

### Requisitos Funcionais vs. Não Funcionais

A classificação clássica divide as necessidades do sistema em dois grandes pilares:

**Requisitos Funcionais (RF):** Descrevem as **interações entre o sistema e seu ambiente**, detalhando os

serviços que o software deve fornecer, como ele deve reagir a entradas e como deve se comportar em situações específicas. Eles representam o comportamento do sistema.

- Exemplo: "O sistema deve permitir o cancelamento de uma consulta em até 24h antes do horário marcado."

**Requisitos Não Funcionais (RNF):** Descrevem restrições e atributos de qualidade do sistema. Em uma plataforma de e-commerce, permitir que o usuário adicione itens ao carrinho é um RF. No entanto, garantir que o sistema suporte 10.000 usuários simultâneos durante a "Black Friday" sem travar é um RNF de Escalabilidade. Se um RNF falha, o sistema pode se tornar inútil, mesmo que todas as funções existam.

- Exemplo: "O sistema deve criptografar todos os dados dos prontuários (Segurança)."

Vamos olhar um pouco melhor com exemplos de Requisitos Funcionais e Não funcionais.

Em uma plataforma de comércio eletrônico (e-commerce), os requisitos funcionais descrevem o comportamento explícito do sistema e as **interações do usuário** com as ferramentas (como o carrinho de compras), enquanto os requisitos não funcionais ditam as **restrições arquiteturais e atributos de qualidade** (FURPS+) que determinam se o sistema operará de forma segura, rápida e confiável sob estresse.

A seguir, é apresentada a Tabela 1.3, exemplificando essas diferenças entre os dois tipos de requisitos

Tabela 1.3: Exemplos de Requisitos para um Comércio Eletrônico

| Categoria                | ID  | Requisito do Sistema                                                                                                                               | Explicação Pedagógica e Técnica                                                                                                                                                    |
|--------------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Requisito Funcional (RF) | RF1 | O sistema deve permitir que o cliente navegue pelo catálogo, adicione, altere a quantidade ou remova itens de produtos de seu carrinho de compras. | <b>O que o sistema faz:</b> Define a operação básica de fluxo de dados de uma transação, manipulando e persistindo temporariamente a intenção de compra do usuário.                |
| Requisito Funcional (RF) | RF2 | O sistema deve exibir as especificações técnicas detalhadas, fotos descritivas e o preço atualizado de cada produto selecionado.                   | <b>O que o sistema faz:</b> É um requisito funcional orientado ao conteúdo, estruturando os objetos de dados dinâmicos extraídos do banco de dados para apresentação na interface. |

Continua na próxima página...

Tabela 1.3 – Continuação da página anterior

| Categoria                                         | ID              | Requisito do Sistema                                                                                                                                                                                                     | Explicação Pedagógica e Técnica                                                                                                                                                                                       |
|---------------------------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requisito Funcional (RF)</b>                   | <b>RF3</b>      | O sistema deve permitir que o cliente insira e salve suas informações de faturamento e endereço de envio durante a finalização da compra ( <i>checkout</i> ).                                                            | <b>O que o sistema faz:</b> Representa a captura e o processamento de entradas de dados externos essenciais para as transações subsequentes de logística.                                                             |
| <b>Requisito Funcional (RNF) – Usabilidade</b>    | <b>Não RNF1</b> | A interface da aplicação deve ser projetada de forma legível, evitando paletas de cores que prejudiquem usuários com daltonismo, e deve oferecer suporte a telas de toque ( <i>touchscreen</i> ) em dispositivos móveis. | <b>Como o sistema deve ser:</b> Foca na ergonomia de uso e nos fatores humanos, garantindo que o sistema seja acessível e de fácil compreensão para diferentes perfis de usuários.                                    |
| <b>Requisito Funcional (RNF) – Confiabilidade</b> | <b>Não RNF2</b> | Em caso de falha de conexão com os serviços externos de cálculo de frete ou de impostos, o sistema deve utilizar uma tabela de taxas locais em cache ( <i>store and forward</i> ) para permitir a conclusão do pedido.   | <b>Como o sistema deve ser:</b> Garante a tolerância a falhas. Segundo Larman, os varejistas não podem interromper suas vendas devido à indisponibilidade de APIs externas de parceiros [5: 131].                     |
| <b>Requisito Funcional (RNF) – Desempenho</b>     | <b>Não RNF3</b> | A API de transação deve autorizar pagamentos com cartão de crédito em um tempo de resposta inferior a 30 segundos em pelo menos 90% das requisições sob carga máxima.                                                    | <b>Como o sistema deve ser:</b> A velocidade é vital para a retenção do cliente. Estudos práticos mostram que acréscimos de milissegundos na latência impactam diretamente a satisfação e reduzem as vendas [5: 132]. |

Continua na próxima página...



Tabela 1.3 – Continuação da página anterior

| Categoria                                          | ID              | Requisito do Sistema                                                                                                                                                                                | Explicação Pedagógica e Técnica                                                                                                                                                                   |
|----------------------------------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requisito Funcional (RNF) – Suportabilidade</b> | <b>Não RNF4</b> | O mecanismo de cálculo de descontos da loja deve ser estruturado através de uma arquitetura modular de regras de negócio plugáveis que possam ser alteradas sem afetar o fluxo principal do código. | <b>Como o sistema deve ser:</b> Trata da facilidade de evolução e manutenção. Permite que as equipes modifiquem as regras de promoções diárias e sazonais de forma dinâmica e ágil [5: 131, 132]. |
| <b>Requisito Funcional (RNF) – Segurança</b>       | <b>Não RNF5</b> | O sistema deve assegurar a confidencialidade e a integridade das senhas e dados financeiros dos usuários por meio do uso de protocolos de criptografia forte.                                       | <b>Como o sistema deve ser:</b> Restrição de segurança essencial para mitigar riscos de invasões ou vazamento de informações confidenciais do cliente [5: 130].                                   |

### O Modelo FURPS+

Utilizado para categorizar requisitos, especialmente os não funcionais. Proposto originalmente pela Hewlett-Packard e amplamente adotado por Larman no Processo Unificado, o acrônimo FURPS+ serve como um guia mnemônico para garantir que nenhuma categoria essencial de requisitos seja esquecida durante a elicitação

- **F (Functionality):** Recursos, segurança.
- **U (Usability):** Fatores humanos, ajuda, documentação.
- **R (Reliability):** Frequência de falhas, recuperabilidade.
- **P (Performance):** Tempo de resposta, uso de recursos.
- **S (Supportability):** Adaptabilidade, facilidade de manutenção.
- **+** : Inclui restrições de implementação, interface, operacionais e legais.

Este modelo transforma "desejos vagos" em metas técnicas mensuráveis, permitindo que o arquiteto de software tome decisões baseadas em atributos de qualidade.

## Requisitos de Usuário vs. Sistema

Vamos detalhar Requisitos de Usuário vs. Sistema, para facilitar a compreensão sobre como a Engenharia de Requisitos organiza as informações para diferentes públicos e finalidades dentro de um projeto de software.

A Engenharia de Requisitos organiza a informação em diferentes camadas para atender a públicos distintos, evitando o ruído na comunicação:

A distinção apresentada na Tabela 1.4 é fundamental para evitar falhas de comunicação e garantir que o sistema entregue atenda às expectativas reais dos envolvidos.

Tabela 1.4: Tipos de requisitos

| Tipo                 | Público-alvo                 | Formato                                                       |
|----------------------|------------------------------|---------------------------------------------------------------|
| Requisito de Usuário | Gerentes e Clientes          | Linguagem natural simples, foco no negócio.                   |
| Requisito de Sistema | Desenvolvedores e Arquitetos | Descrição técnica detalhada, serve como base para o contrato. |

A seguir, explicamos os dois níveis de abstração propostos:

- **Requisitos de Usuário: O Foco no Negócio**

Os requisitos de usuário representam o nível mais alto de abstração da especificação. São declarações em linguagem natural e diagramas intuitivos sobre os serviços que o sistema deve prestar e suas restrições operacionais. Eles são escritos para clientes e gerentes que precisam de uma visão de alto nível do negócio

- **Público-alvo:** São escritos principalmente para os gerentes de negócio e clientes, que precisam entender o que o sistema fará para apoiar seus processos, sem entrar em detalhes técnicos.
- **Formato e Linguagem:** Devem ser redigidos em linguagem natural simples, complementada por diagramas intuitivos. O objetivo aqui é descrever os serviços esperados e as restrições operacionais de forma clara para quem não é especialista em tecnologia.
- **Exemplo na Clínica:** "O sistema deve permitir que o nutricionista visualize o histórico de peso do paciente nos últimos seis meses".

- **Requisitos de Sistema: A Ponte para a Implementação**

Os requisitos de sistema são versões expandidas e detalhadas dos requisitos de usuário. São descrições detalhadas e técnicas das funções e serviços. Eles servem como a base do "contrato" de desenvolvimento e são destinados aos arquitetos e desenvolvedores, fornecendo a precisão necessária

para a codificação.

- **Público-alvo:** Destinam-se aos desenvolvedores, engenheiros e arquitetos de software, fornecendo a precisão necessária para iniciar o projeto técnico e a codificação.
- **Formato e Linguagem:** Utilizam uma descrição técnica detalhada, muitas vezes acompanhada de modelos de sistema (como diagramas de classes ou sequência). Eles definem exatamente o que deve ser implementado e servem como base para o contrato legal entre o cliente e a equipe de desenvolvimento.
- **Exemplo na Clínica:** "Ao acionar a função 'Ver Histórico', o sistema deve realizar uma consulta na tabela Evolucao\_Clinica filtrando pelo ID\_Paciente e ordenando os registros de forma decrescente pela data".

### Por que essa separação é importante?

**Analogia Pedagógica:** Pense em uma obra de construção civil. O Requisito de Usuário seria o pedido do dono da casa: "Quero uma sala ampla e bem iluminada". O Requisito de Sistema seria a planta técnica do arquiteto: "A sala terá 40m<sup>2</sup>, com uma janela de vidro temperado de 3m por 2m voltada para o norte".

Conforme destacado por Sommerville, a imprecisão na especificação é uma das maiores causas de atrasos e custos extras em projetos.

- Se o documento for **técnico demais**, o cliente não conseguirá validar se os requisitos estão corretos.
- Se for **vago demais**, o desenvolvedor poderá interpretar o requisito de uma forma que simplifique a programação, mas que não resolva o problema do usuário.

Portanto, a Tabela 1.4 demonstra que a Engenharia de Software não busca apenas documentar funções, mas sim **traduzir necessidades de negócio em especificações técnicas precisas**, garantindo a qualidade e a utilidade final do produto.

No contexto da Engenharia de Requisitos, um **stakeholder** (ou parte interessada) é definido como *qualquer pessoa ou grupo que tenha um interesse direto no sucesso do projeto ou que seja afetado pelo sistema de forma direta ou indireta*.

A seguir, são detalhadas as características e os papéis dos stakeholders conforme as fontes

#### 1. Definições Principais

- Segundo Roger Pressman: É qualquer pessoa que tenha uma "aposta"(stake) no resultado bem-sucedido do projeto, incluindo gerentes de negócio, usuários finais, engenheiros de software e pessoal de suporte.
- Segundo Ian Sommerville: É uma pessoa ou um papel (role) afetado pelo sistema de alguma

maneira. O autor destaca que os stakeholders possuem influências diretas ou indiretas sobre os requisitos do sistema.

2. **Exemplos de Papéis de Stakeholders** Os stakeholders podem desempenhar diversas funções dentro e fora da organização:

- **Especialistas de Domínio:** Pessoas que entendem profundamente o negócio (ex: o nutricionista na Clínica de Bem-estar).
- **Usuários Finais:** Aqueles que realmente operarão o software para atingir um objetivo de negócio.
- **Clientes:** Quem solicitou o software originalmente e, geralmente, coordena o financiamento do projeto.
- **Gerentes (Sêniores e de Projeto):** Responsáveis por definir questões de negócio e planejar/controlar o desenvolvimento.
- **Equipe Técnica:** Engenheiros de software, arquitetos, designers de UI/UX, testadores e analistas.
- **Stakeholders Externos:** Reguladores governamentais ou agências fiscais que podem impor restrições ao sistema.

3. **Importância na Engenharia de Requisitos** A identificação correta dos stakeholders é uma das tarefas iniciais da atividade de concepção (inception). Isso é crucial porque:

- **Múltiplas Perspectivas:** Cada stakeholder tem um ponto de vista diferente, o que gera requisitos que podem ser inconsistentes ou conflitantes entre si.
- **Descoberta de Requisitos:** Durante a elicitação, o analista deve perguntar a cada stakeholder: "Quem mais você acha que eu deveria consultar?", permitindo que a lista de envolvidos cresça e cubra todas as necessidades do sistema.
- **Interesses e Contratos:** Segundo Craig Larman, um Caso de Uso pode ser visto como um contrato entre stakeholders, onde o sistema deve capturar comportamentos que satisfaçam os interesses de todos os envolvidos.

Em resumo, os **stakeholders** são a fonte primária de informação para a Engenharia de Requisitos, e o sucesso do projeto depende do alinhamento entre as expectativas dessas partes e a solução técnica construída.

### 1.2.3 O olhar do Engenheiro de Software

**Não seja um mero "Tradutor":** Muitos analistas acham que seu trabalho é ouvir o cliente e "traduzir" para o técnico. Khononov alerta que o entendimento do desenvolvedor é o que vai para a produção, não o conhecimento do especialista. Portanto, o engenheiro deve ser um co-criador do modelo. Se você aceita requisitos sem entender o "porquê" do negócio, você corre o risco de automatizar processos ineficientes ou errados.

### 1.2.4 Estudo de Caso Integrador: Clínica de Saúde (Etapa 1.2)

#### Linguagem Ubíqua na Clínica:

- Após conversas, definimos que "Sessão" é o termo para um atendimento individual de massoterapia.
- Não usaremos "Consulta", pois os especialistas não são médicos.

#### Identificação de Requisitos:

- RF1: O sistema deve registrar o histórico de alergias do Paciente no Prontuário Digital.
- RNF1 (FURPS+ Reliability): O sistema deve realizar backup automático dos dados a cada 24 horas.
- RNF2 (Usability): A interface deve permitir o agendamento de uma sessão em no máximo 3 cliques.

#### IA como Parceira:

*"Aja como um Engenheiro de Requisitos experiente. Para o domínio de uma Clínica de Bem-estar, proponha 3 Requisitos Não Funcionais críticos seguindo o modelo FURPS+, justificando cada um com base em riscos de negócio."*

O papel do engenheiro é extrair o conhecimento essencial desses especialistas.

O domínio é a área de conhecimento ou atividade específica em torno da qual o software é construído. Por exemplo, em um sistema bancário, o domínio envolve contas, transferências e juros. Especialistas de Domínio: São as pessoas que entendem profundamente o negócio (ex: o nutricionista na clínica), mas que não necessariamente entendem de software. O papel do engenheiro é extrair o conhecimento essencial desses especialistas.

### 1.2.5 Exercícios de Fixação

(Para realizar em papel e caneta, sem consulta, para treinar para a prova)

1. **Conceitual:** Diferencie Requisitos Funcionais (RF) de Requisitos Não Funcionais (RNF). Por que, em muitos casos, a falha em um RNF pode ser considerada mais crítica do que a falha em um RF?

2. **Linguagem Ubíqua:** No contexto de Domain-Driven Design (DDD), o que define a Linguagem Ubíqua e qual sua importância para a comunicação entre técnicos e especialistas de domínio?
3. **Comparativo:** Apresente as diferenças fundamentais entre Requisitos de Usuário e Requisitos de Sistema quanto ao seu público-alvo e nível de detalhamento.

4. **Cenário (Clínica):** Considere o estudo de caso da Clínica de Bem-estar.

Classifique os requisitos abaixo como RF ou RNF (especificando a categoria FURPS+ para o RNF):

- a) O sistema deve realizar backup automático dos dados a cada 24 horas.
  - b) O sistema deve registrar o histórico de alergias do Paciente.
  - c) O acesso aos prontuários deve ser restrito ao profissional que realiza o atendimento.
  - d) O tempo de resposta para a busca de pacientes deve ser inferior a 2 segundos.
5. **Reflexão:** Explique o "Efeito Telefone Sem Fio" na tradução de requisitos e como a postura do engenheiro de software como "co-criador do modelo" ajuda a mitigar esse problema.

6. **Questão ENADE:** A Engenharia de Requisitos é um processo disciplinado que atua como ponte entre as necessidades dos stakeholders e a precisão exigida pela construção de software. Sobre esse processo e a classificação de requisitos, avalie as afirmações a seguir.

I. Requisitos Não Funcionais, como confiabilidade e segurança, raramente afetam a arquitetura global do sistema, sendo tratados apenas na fase de codificação final.

II. Segundo Eric Evans, a Linguagem Ubíqua deve ser utilizada de forma consistente em todas as formas de comunicação da equipe e refletida obrigatoriamente no código-fonte.

III. O Modelo FURPS+ é uma técnica utilizada para categorizar requisitos, onde o "S" refere-se à Suportabilidade, abrangendo aspectos de adaptabilidade e facilidade de manutenção.

É correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

## 1.3 Considerações Finais

Ao concluir esta primeira unidade, você compreendeu que a modelagem não se resume à criação de diagramas, mas sim a um processo intelectual de **abstração** fundamental para gerenciar a complexidade de sistemas reais. Aprendemos a distinguir tecnicamente a **Análise**, focada em investigar o domínio do problema para "fazer a coisa certa", da atividade de **Projeto (Design)**, que se dedica a conceber a solução técnica para "fazer a coisa da maneira certa". Essa base sólida permite que você atue como um investigador do negócio, garantindo que o modelo seja uma representação simplificada, porém precisa, das necessidades do cliente.

Além disso, exploramos como a **Engenharia de Requisitos** e a **Modelagem de Domínio** servem como pontes de comunicação entre os stakeholders e a equipe técnica. Vimos que a construção de uma **Linguagem Ubíqua (DDD)** é o recurso mais poderoso para eliminar o "efeito telefone sem fio", garantindo que o conhecimento dos especialistas de negócio seja refletido fielmente nos artefatos de software. Ao classificar requisitos em funcionais e não funcionais por meio do modelo **FURPS+**, estabelecemos os critérios de qualidade e as restrições que nortearão a viabilidade de todo o projeto.

Este entendimento profundo sobre o domínio e as necessidades de negócio é o alicerce indispensável para avançarmos rumo à **Unidade 2 - Conceitos de Orientação a Objetos**. No próximo capítulo, utilizaremos o conhecimento extraído até aqui para começar a dar forma estrutural ao sistema, transformando conceitos de negócio em **classes, objetos e responsabilidades**. Lembre-se: um bom design orientado a objetos nasce da capacidade de reduzir o "hiato de representação" entre o mundo real e o código, o que só é possível quando a análise de domínio foi realizada com rigor.



# Unidade 2 - Conceitos de Orientação a Objetos



## 2. Orientação a Objetos e Descoberta de Classes

### Conteúdo da Unidade

|       |                                                                              |    |
|-------|------------------------------------------------------------------------------|----|
| 2.1   | Introdução à Unidade                                                         | 30 |
| 2.2   | Fundamentos de Objetos e Descoberta de Classes                               | 31 |
| 2.2.1 | Classes vs. Objetos: O Molde e a Instância                                   | 31 |
| 2.2.2 | Atributos e Responsabilidades (Saber vs. Fazer)                              | 33 |
| 2.2.3 | Os Pilares da OO Aplicados à Modelagem                                       | 34 |
| 2.3   | Diagramas da UML                                                             | 36 |
| 2.4   | Diagrama de Classes                                                          | 37 |
| 2.5   | Como Descobrir Classes Conceituais: O Método Larman                          | 40 |
| 2.6   | O Olhar do Engenheiro de Software: Hiato de Representação e Modelos Anêmicos | 43 |
| 2.7   | Estudo de Caso Integrador: Clínica de Saúde (Etapa 2)                        | 43 |
| 2.8   | Guia Prático: Engenharia de Prompts para Descoberta de Classes               | 43 |
| 2.8.1 | Princípios da Modelagem Assistida por IA                                     | 44 |
| 2.8.2 | Templates de Prompts Estruturados                                            | 44 |
| 2.8.3 | Roteiro Prático de Trabalho para as Equipes                                  | 47 |
| 2.9   | Exercícios de Fixação                                                        | 48 |



**Objetivo:** Desenvolver a capacidade de identificar e representar os principais elementos de um domínio de negócio utilizando os princípios da Orientação a Objetos. Ao final da unidade, o estudante será capaz de identificar classes, objetos, responsabilidades e relacionamentos, organizando essas informações em modelos UML coerentes e utilizando ferramentas de Inteligência Artificial para apoiar a descoberta, validação e refinamento dos modelos produzidos

Depois de compreender o problema que precisa ser resolvido, o próximo desafio consiste em organizar o conhecimento adquirido em uma estrutura capaz de representar o domínio da aplicação. A Orientação a Objetos oferece um modelo conceitual amplamente utilizado para representar entidades, responsabilidades e relacionamentos presentes em sistemas de informação. A UML, por sua vez, fornece uma linguagem padronizada para comunicar essas soluções entre analistas, desenvolvedores e demais envolvidos no projeto.

Nesta unidade serão estudados os fundamentos da Orientação a Objetos, os principais elementos da UML e a forma como esses conceitos podem ser utilizados para representar diferentes perspectivas de um sistema. Também será explorado o uso da Inteligência Artificial como apoio à identificação de entidades, responsabilidades e relacionamentos, destacando a necessidade de validação humana em todas as etapas do processo de modelagem.

Ao final deste capítulo, você será capaz de:

- Compreender a transição paradigmática entre o modelo mental de domínio e as classes de software;
- Identificar classes conceituais utilizando a lista de categorias e a análise linguística;
- Aplicar de forma prática e rigorosa os pilares da OO (encapsulamento, herança e polimorfismo) no modelo de análise;
- Reduzir o Hiato de Representação de forma prática no estudo de caso integrador.

### 2.1 Introdução à Unidade

A transição para a Orientação a Objetos (OO) exige o que Martin Fowler conceitua como uma “reconfiguração cerebral” (*brain rewiring*). Para desenvolvedores habituados ao estilo procedural clássico, onde dados e funções residem de forma separada no código, estruturar o software como uma comunidade de objetos que colaboram e distribuem responsabilidades lógicas representa um verdadeiro salto paradigmático.

A grande vantagem desse paradigma reside no fato de que ele permite modelar o software de maneira

que este reflita, de forma muito próxima, as estruturas do mundo real. Se dominarmos essa habilidade, construiremos sistemas robustos, altamente adaptáveis a mudanças de negócio e com o mínimo possível de distorção semântica.

## 2.2 Fundamentos de Objetos e Descoberta de Classes

### 2.2.1 Classes vs. Objetos: O Molde e a Instância

No paradigma orientado a objetos, a modelagem de domínio é estruturada sob dois conceitos fundamentais que se inter-relacionam dinamicamente durante a execução do software: as classes e os objetos.

#### O Conceito de Classe

Uma **classe** constitui a especificação abstrata, estática e conceitual do domínio. Ela atua como um molde, gabarito estrutural ou planta (*blueprint*) que descreve e define um conjunto de elementos com características (atributos) e comportamentos (operações) idênticos.

- **Abstração de Software:** Sob a perspectiva da Engenharia de Software, as classes atuam como tipos de dados abstratos cujo estado interno é privado e só pode ser alterado por intermédio de operações públicas e autorizadas.
- **Definição do Tipo:** A classe especifica quais dados (atributos) o objeto reterá e quais responsabilidades de comportamento (operações/métodos) ele poderá executar, servindo como o contrato estático do sistema.

#### O Conceito de Objeto

Um **objeto** (ou instância), por sua vez, é uma entidade dinâmica e concreta que reside na memória RAM durante a execução do programa. Ele encapsula um estado local e responde a estímulos temporais através da troca de mensagens com outros objetos.

- **Identidade do Objeto:** Cada objeto possui uma identidade única e independente dos valores lógicos que seus atributos possam assumir no momento. Dois objetos da classe *Paciente* podem ter exatamente o mesmo nome, idade e endereço, mas ainda assim serão distinguidos em memória por suas respectivas identidades lógicas de objeto.
- **Instanciação Dinâmica:** Quando um especialista de domínio realiza uma ação real (ex: registrar o paciente “Carlos Silva”), o sistema cria (instancia) um objeto concreto em memória, preenchendo as variáveis de instância com os dados do mundo real.

Fazendo assim uma analogia com a classe e os objetos. Um objeto, por sua vez, é uma instância dinâmica

e concreta desse molde que encapsula um estado local e responde a estímulos temporais através de mensagens. A Figura 2.1 mostra de forma lúdica como funcionam as formas (classes) e os bichinhos que são produzidos pela forma.



Figura 2.1: Classes como moldes

### Comunicação por Mensagens

Os objetos de um sistema de software não trabalham isoladamente; eles cooperam de forma distribuída para implementar as regras de negócio. Essa cooperação se dá estritamente por meio de **Passagem de Mensagens** (*message passing*).

- **Funcionamento:** O objeto remetente (*SenderObject*) envia uma mensagem (estímulo) estruturada ao objeto receptor (*ReceiverObject*) na forma de mensagem (parâmetros). A mensagem é essencialmente uma chamada de procedimento ou requisição de serviço.
- **Resposta do Receptor:** Ao receber o estímulo, o objeto receptor busca em sua definição de classe a operação que implementa o nome da mensagem (o algoritmo do método correspondente), executa o código sob o seu próprio estado encapsulado e, finalmente, devolve o fluxo de controle (e opcionalmente um valor de retorno) para o chamador.
- **Proteção e Encapsulamento:** Nenhuma classe externa pode alterar diretamente os atributos privados de outro objeto. Toda alteração deve passar pela muralha de métodos públicos, onde o próprio objeto valida a consistência de seus dados em resposta à mensagem recebida.

### Exemplo de Representação UML de Classe vs. Objeto

Para ilustrar claramente como esses conceitos se materializam na documentação de Engenharia de Software, a UML estabelece notações específicas para diferenciar a visão estática (Classe) da visão

dinâmica de instâncias em tempo de execução (Objeto).

Tabela 2.1: Representação UML de Classe vs. Objeto em Memória

| UML Class Box (Classe / Molde)                                                                       | UML Object Box (Objeto / Instância)                                                                                         |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>SessaoAtendimento</b>                                                                             | <b><u>sessao1: SessaoAtendimento</u></b>                                                                                    |
| - dataAtendimento: Date<br>- horarioInicio: Time<br>- status: String<br>- valorSessao: Money         | dataAtendimento = 2026-08-30<br>horarioInicio = 14:00<br>status = "Confirmado"<br>valorSessao = R\$ 150.00                  |
| + confirmarAtendimento(): void<br>+ calcularValorFinal(): Money<br>+ validarChoqueHorario(): boolean | <i>(Estado interno e atributos privados<br/>           protegidos por uma muralha<br/>           de operações públicas)</i> |

#### Diferenças técnicas na representação acima:

1. **Sublinhado mandatório:** Na caixa do Objeto, o nome da instância e o nome da classe são obrigatoriamente sublinhados (nomeInstancia:NomeClasse). Isso indica que se trata de uma especificação de instância (um registro dinâmico) e não do tipo estático.
2. **Atribuição de Valores:** Enquanto a Classe define os nomes e tipos genéricos das variáveis (Date, Time), o Objeto exibe os valores literais atribuídos em memória para aquele cenário (2026-08-30, 14:00).
3. **Ausência de métodos na caixa de instâncias:** Como o comportamento é comum a todas as instâncias e é armazenado na definição da classe, o compartimento de operações/métodos é omitido na representação do objeto, expondo apenas os dados dinâmicos do seu estado.

### 2.2.2 Atributos e Responsabilidades (Saber vs. Fazer)

Segundo Craig Larman, o segredo da modelagem de classes está no conceito de **Responsabilidade**. Uma responsabilidade é um contrato ou obrigação de uma classe, classificada sob duas perspectivas fundamentais:

1. **Responsabilidade de Conhecer (Knowing):** O que o objeto sabe sobre si mesmo ou sobre os objetos relacionados, incluindo dados encapsulados e conexões estruturais.
2. **Responsabilidade de Fazer (Doing):** O que o objeto faz, as ações que ele inicia ou as regras de negócio que ele executa diretamente, incluindo controlar e coordenar atividades em outros objetos.

### 2.2.3 Os Pilares da OO Aplicados à Modelagem

#### Encapsulamento e a Muralha de Métodos

O encapsulamento agrupa dados e procedimentos sob uma mesma identidade semântica. Roger Pressman ilustra esse conceito por meio de uma metáfora clássica: os atributos privados de um objeto residem protegidos dentro de uma “muralha” composta pelas suas próprias operações públicas. Nenhuma classe externa pode alterar esses dados diretamente; a comunicação ocorre estritamente por passagem de mensagens (*message passing*). Esse isolamento aumenta drasticamente a coesão interna e reduz o acoplamento global, evitando que mudanças pontuais na implementação gerem um efeito cascata de erros no código. Veja a representação deste conceito na Figura 2.2.

#### Herança e Conformidade Conceitual

A herança permite que uma subclasse (especialização) herde automaticamente todos os atributos e métodos de sua superclasse (generalização). Contudo, na modelagem profissional, herança não deve ser usada apenas por mero atalho técnico ou preguiça de digitar código. Craig Larman estabelece duas diretrizes mandatórias para validar uma herança legítima:

- **A Regra dos 100% (Conformidade de Definição):** A subclasse deve conformar-se estruturalmente a 100% da definição da superclasse, incluindo todos os seus atributos e associações lógicas.
- **A Regra do “É Um” (Is-A - Conformidade de Conjunto):** Todos os membros da subclasse devem pertencer logicamente ao conjunto definido pela superclasse. O teste linguístico é simples: “A subclasse É UMA superclasse?” (ex: um Massoterapeuta é um Especialista).

Veja a representação deste conceito na Figura 2.2.



Figura 2.2: Encapsulamento e Herança

### Polimorfismo e Componentes Plugáveis

O polimorfismo é a capacidade de subclasses responderem à mesma mensagem abstrata de formas distintas e especializadas. Ao modelar por polimorfismo, o engenheiro elimina estruturas condicionais complexas do tipo `if-then-else` ou `switch-case`. Em vez de ter uma classe de controle cheia de validações manuais, definimos que cada tipo de consulta ou atendimento executa sua própria regra de cálculo ao receber a mensagem `calcularValor()`. Veja a representação deste conceito na Figura 2.3.



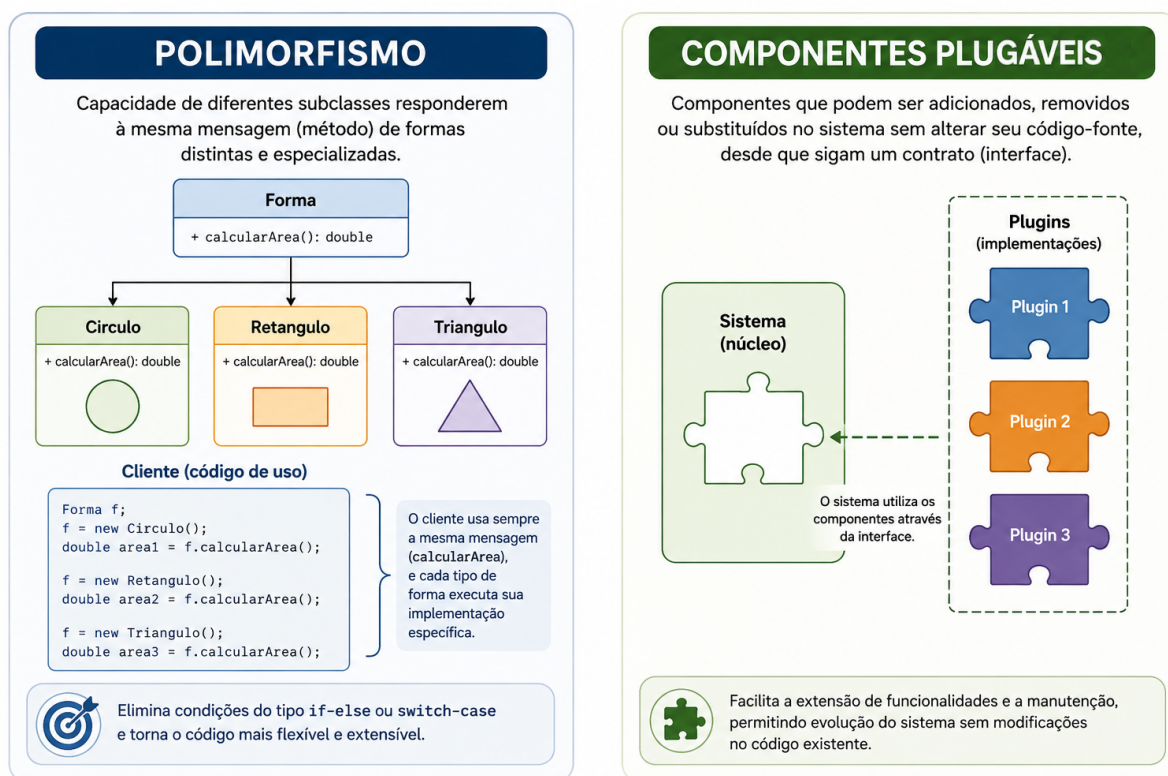


Figura 2.3: Polimorfismo e Componentes

## 2.3 Diagramas da UML

A UML (Unified Modeling Language) é uma família de notações gráficas, apoiada por um metamodelo único, utilizada para especificar, visualizar, construir e documentar artefatos de sistemas de software, especialmente aqueles construídos sob o paradigma de Orientação a Objetos.

A especificação da UML 2 define oficialmente 13 tipos de diagramas. No entanto, a linguagem não impõe uma metodologia rígida de desenvolvimento e permite omitir ou suprimir detalhes de acordo com o contexto lúdico ou técnico do analista. De forma geral, esses diagramas são divididos em duas grandes categorias: os Diagramas Estruturais (focados no aspecto estático do sistema) e os Diagramas Comportamentais (focados na dinâmica e na execução do software).



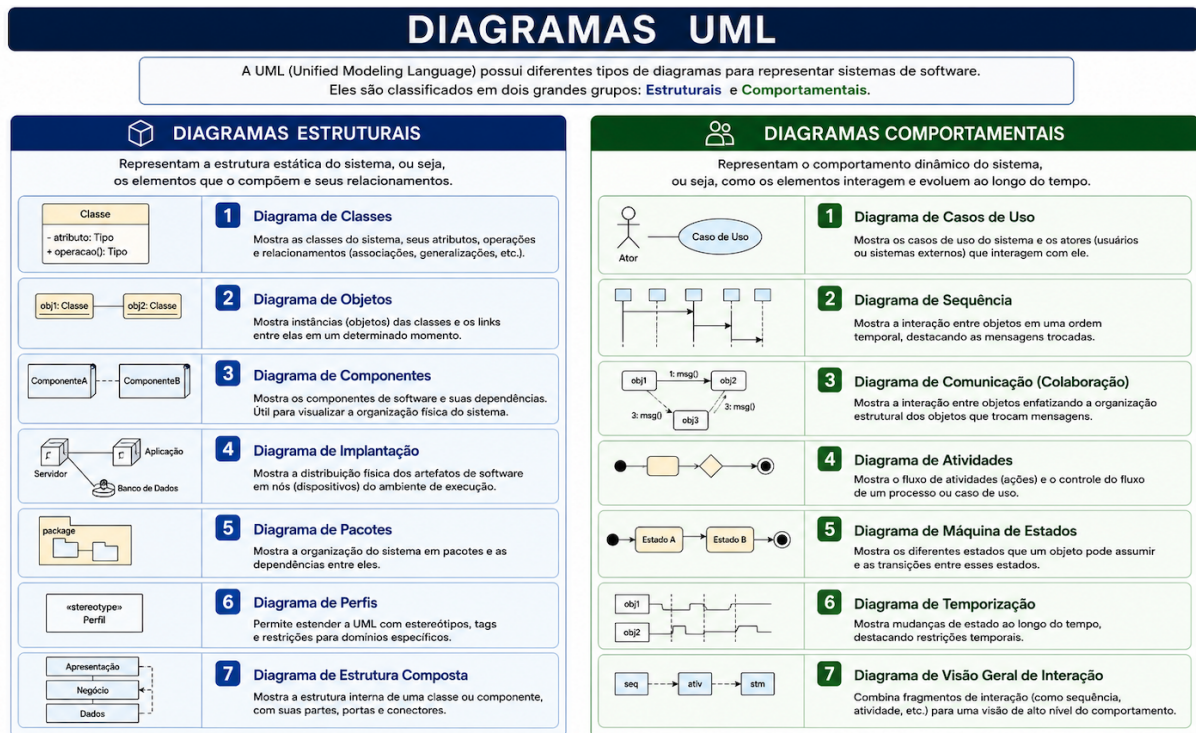


Figura 2.4: Diagramas da UML

Agora vamos ver o diagrama de classes.

## 2.4 Diagrama de Classes

O **Diagrama de Classes** é o modelo estrutural estático mais importante e amplamente utilizado, funcionando como a espinha dorsal de todo o processo de análise e design orientado a objetos. Sob a ótica de Martin Fowler, sua finalidade primária é ilustrar os tipos de objetos presentes no sistema e as diversas formas de relacionamentos estáticos que os conectam.

Na modelagem profissional e acadêmica, é fundamental distinguir as duas principais perspectivas de uso do diagrama:

- 1. Perspectiva Conceitual (Modelo de Domínio):** O diagrama descreve coisas e conceitos do mundo real do domínio de interesse, de forma totalmente independente de qualquer linguagem de programação ou infraestrutura tecnológica.
- 2. Perspectiva de Software (Design Class Diagram - DCD):** O diagrama especifica as classes de programação reais, seus atributos privados com tipos definidos, assinaturas completas de métodos com parâmetros e retornos, e visibilidades operacionais.

## O Conceito de Classe

Uma **classe** constitui a especificação abstrata, estática e conceitual do domínio. Ela atua como um molde, gabarito estrutural ou planta (*blueprint*) que descreve e define um conjunto de elementos com características (atributos) e comportamentos (operações) idênticos.

- **Abstração de Software:** Sob a perspectiva da Engenharia de Software, as classes atuam como tipos de dados abstratos cujo estado interno é privado e só pode ser alterado por intermédio de operações públicas e autorizadas.
- **Definição do Tipo:** A classe especifica quais dados (atributos) o objeto reterá e quais responsabilidades de comportamento (operações/métodos) ele poderá executar, servindo como o contrato estático do sistema.

## O Conceito de Objeto

Um **objeto** (ou instância), por sua vez, é uma entidade dinâmica e concreta que reside na memória RAM durante a execução do programa. Ele encapsula um estado local e responde a estímulos temporais através da troca de mensagens com outros objetos.

- **Identidade do Objeto:** Cada objeto possui uma identidade única e independente dos valores lógicos que seus atributos possam assumir no momento. Dois objetos da classe *Paciente* podem ter exatamente o mesmo nome, idade e endereço, mas ainda assim serão distinguidos em memória por suas respectivas identidades lógicas de objeto.
- **Instanciação Dinâmica:** Quando um especialista de domínio realiza uma ação real (ex: registrar o paciente “Carlos Silva”), o sistema cria (instancia) um objeto concreto em memória, preenchendo as variáveis de instância com os dados do mundo real.

## Comunicação por Mensagens

Os objetos de um sistema de software não trabalham isoladamente; eles cooperam de forma distribuída para implementar as regras de negócio. Essa cooperação se dá estritamente por meio de **Passagem de Mensagens** (*message passing*).

- **Funcionamento:** O objeto remetente (*SenderObject*) envia uma mensagem (estímulo) estruturada ao objeto receptor (*ReceiverObject*) na forma de mensagem(parâmetros). A mensagem é essencialmente uma chamada de procedimento ou requisição de serviço.
- **Resposta do Receptor:** Ao receber o estímulo, o objeto receptor busca em sua definição de classe a operação que implementa o nome da mensagem (o algoritmo do método correspondente),

executa o código sob o seu próprio estado encapsulado e, finalmente, devolve o fluxo de controle (e opcionalmente um valor de retorno) para o chamador.

- **Proteção e Encapsulamento:** Nenhuma classe externa pode alterar diretamente os atributos privados de outro objeto. Toda alteração deve passar pela muralha de métodos públicos, onde o próprio objeto valida a consistência de seus dados em resposta à mensagem recebida.

### Exemplo de Representação UML de Classe vs. Objeto

Para ilustrar claramente como esses conceitos se materializam na documentação de Engenharia de Software, a UML estabelece notações específicas para diferenciar a visão estática (Classe) da visão dinâmica de instâncias em tempo de execução (Objeto).

Tabela 2.2: Representação UML de Classe vs. Objeto em Memória

| UML Class Box (Classe / Molde)                                                                       | UML Object Box (Objeto / Instância)                                                                                         |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>SessaoAtendimento</b>                                                                             | <b><u>sessao1: SessaoAtendimento</u></b>                                                                                    |
| - dataAtendimento: Date<br>- horarioInicio: Time<br>- status: String<br>- valorSessao: Money         | dataAtendimento = 2026-08-30<br>horarioInicio = 14:00<br>status = "Confirmado"<br>valorSessao = R\$ 150.00                  |
| + confirmarAtendimento(): void<br>+ calcularValorFinal(): Money<br>+ validarChoqueHorario(): boolean | <i>(Estado interno e atributos privados<br/>           protegidos por uma muralha<br/>           de operações públicas)</i> |

#### Diferenças técnicas na representação acima:

1. **Sublinhado mandatório:** Na caixa do Objeto, o nome da instância e o nome da classe são obrigatoriamente sublinhados (nomeInstancia:NomeClasse). Isso indica que se trata de uma especificação de instância (um registro dinâmico) e não do tipo estático.
2. **Atribuição de Valores:** Enquanto a Classe define os nomes e tipos genéricos das variáveis (Date, Time), o Objeto exibe os valores literais atribuídos em memória para aquele cenário (2026-08-30, 14:00).
3. **Ausência de métodos na caixa de instâncias:** Como o comportamento é comum a todas as instâncias e é armazenado na definição da classe, o compartimento de operações/métodos é omitido na representação do objeto, expondo apenas os dados dinâmicos do seu estado.

## 2.5 Como Descobrir Classes Conceituais: O Método Larman

A identificação das classes corretas do domínio exige do analista uma investigação estruturada. Craig Larman propõe duas técnicas altamente eficazes para esse mapeamento:

### Método 1: Lista de Categorias de Classes Conceituais

Consiste em varrer a documentação de requisitos analisando o domínio de forma sistemática por categorias consolidadas. O analista deve mapear:

- **Transações de Negócio:** Eventos vitais, geralmente envolvendo dinheiro ou registros operacionais (ex: Agendamento, Sessao, Pagamento).
- **Itens de Linha de Transações:** Detalhes de um evento (ex: ItemDeServico).
- **Papéis de Pessoas:** Funções desempenhadas por usuários ou atores (ex: Especialista, Paciente).
- **Lugares ou Estruturas:** Locais físicos e instalações necessárias (ex: SalaDeAtendimento).

### Método 2: Análise Linguística (Sintagmas Nominais)

Proposta originalmente por Abbott, consiste no mapeamento gramatical direto da descrição dos cenários de uso:

- Os **substantivos** ou frases nominais são convertidos em candidatas a classes conceituais ou atributos.
- Os **verbos** ou frases verbais são convertidos em candidatas a operações (métodos) lógicos.

Agora vamos ver o exemplo de um diagrama de classes. A Figura 2.5 apresenta um exemplo de um diagrama de classe. Veja que as classes aparecem completas com identificação, atributos e métodos.

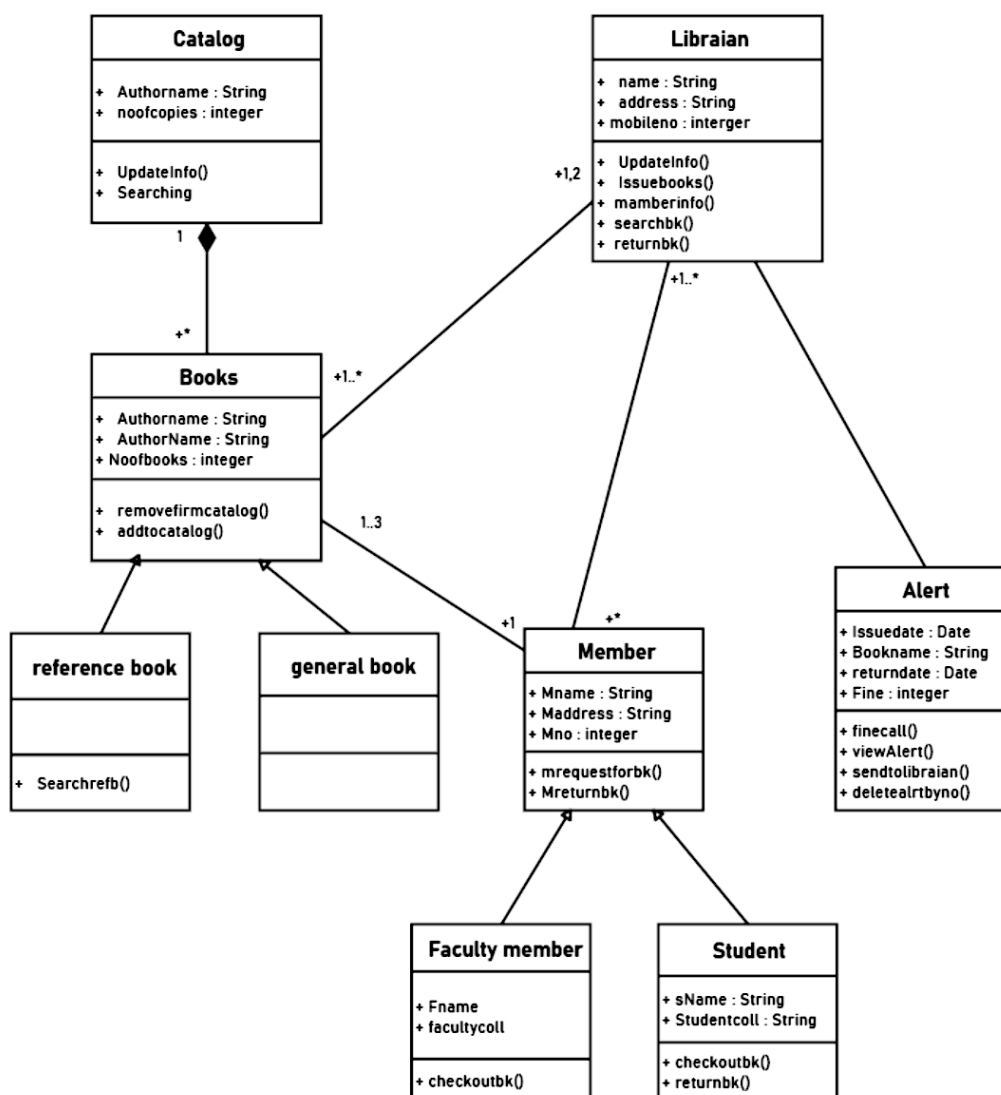


Figura 2.5: Exemplo de Diagrama de Classe

Além disso, existem os relacionamentos entre as classes. Também é possível observar a multiplicidade. A Figura 2.6 apresenta os diferentes tipos de relacionamentos que um diagrama de classes pode ter. Cada relacionamento tem uma notação diferente e representa um propósito diferente também. Verifique o diagrama de classes da Figura 2.5 e todos os relacionamentos e procure a indicação do propósito de cada tipo de relacionamento na Figura 2.6.



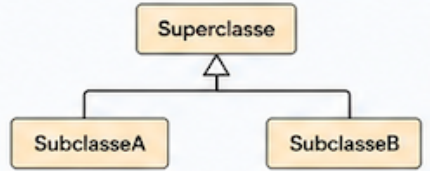
| Tipos de Relacionamentos entre Classes                                                                                                                                                                                                         |                                                                                      |                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| TIPO DE RELACIONAMENTO                                                                                                                                                                                                                         | NOTAÇÃO UML                                                                          | DESCRIÇÃO                                                                               |
| <b>Associação</b><br>Relacionamento estrutural entre classes. Indica que um objeto de uma classe conhece ou usa objetos de outra classe.                                                                                                       |    | É o relacionamento mais geral. Representado por uma linha sólida entre as classes.      |
| <b>Associação Unidirecional</b><br>Uma classe conhece ou usa a outra, mas não no sentido inverso.                                                                                                                                              |    | Representado por uma linha sólida com uma seta indicando a direção da navegação.        |
| <b>Associação Bidirecional</b><br>As classes se conhecem ou usam mutuamente.                                                                                                                                                                   |    | Representado por uma linha sólida com setas nas duas extremidades.                      |
| <b>Agregação (Compartilhamento)</b><br>Relação "todo–parte" fraca. As partes podem existir independentemente do todo.                                                                                                                          |   | Representado por uma linha sólida com um losango branco no lado do "todo".              |
| <b>Composição (Composição Forte)</b><br>Relação "todo–parte" forte. As partes dependem do todo para existir.                                                                                                                                   |  | Representado por uma linha sólida com um losango preto no lado do "todo".               |
| <b>Generalização (Herança)</b><br>Indica uma relação de especialização/generalização ("é um tipo de").                                                                                                                                         |  | Representado por uma linha sólida com um triângulo branco apontando para a superclasse. |
| <b>Dependência</b><br>Uma classe depende temporariamente de outra (classe usada como parâmetro, variável local, etc.).                                                                                                                         |  | Representado por uma linha tracejada com uma seta.                                      |
|  <b>DICA:</b> A escolha do tipo de relacionamento correto é essencial para representar adequadamente as regras de negócio e garantir a qualidade do modelo. |                                                                                      |                                                                                         |

Figura 2.6: Tipos de Relacionamentos entre as Classes

## 2.6 O Olhar do Engenheiro de Software: Hiato de Representação e Modelos Anêmicos

O analista júnior frequentemente comete o erro de projetar classes anêmicas (*Anemic Domain Model*), que são meros contêineres de dados sem comportamento inteligente ou regra de negócio encapsulada (possuindo apenas campos privados e métodos de leitura/escrita de dados). Isso sabota os pilares da OO, relegando toda a inteligência do sistema para scripts soltos.

O objetivo do engenheiro de software de alto nível é atingir o **Baixo Hiato de Representação (Low Representational Gap - LRG)**. LRG significa assegurar que a correspondência entre como pensamos no mundo real (o modelo de domínio) e como codificamos o software seja o mais direta e contínua possível. Se o especialista de nossa clínica fala em “Prontuário”, essa exata palavra deve se materializar como uma classe em nossos diagramas e, futuramente, no código-fonte, combatendo o efeito telefone sem fio.

## 2.7 Estudo de Caso Integrador: Clínica de Saúde (Etapa 2)

Na etapa anterior, identificamos os requisitos e a linguagem ubíqua da nossa clínica de nutrição e massoterapia, definindo que atendimentos são categorizados como “Sessões” e que não usaremos termos médicos como “consultas”. Agora, daremos o passo evolutivo estrutural: a transição das ideias para as classes lógicas do software.

Ao final desta unidade, você deve ter identificado todas as classes do sistema estudo de caso que você está trabalhando. Após a identificação dos objetos, construir as classes e finalizar com o diagrama de classes completo. Veja que o seu diagrama deve ficar semelhante a esse da Figura 2.5

Agora veja que todas as classes que devem ser identificadas no sistema que vocês estão construindo devem seguir esse padrão. Além disso, devem também apresentar os relacionamentos entre as classes, como definido na Seção 2.4.

## 2.8 Guia Prático: Engenharia de Prompts para Descoberta de Classes

Este guia prático foi desenvolvido para orientar os acadêmicos no uso crítico e responsável de ferramentas de Inteligência Artificial Generativa (como Gemini, ChatGPT ou Claude) durante o processo de modelagem de sistemas. O objetivo é utilizar a IA como um copiloto para acelerar a descoberta de classes

conceituais, atributos e responsabilidades, aplicando os métodos clássicos de Craig Larman e a análise gramatical de Abbott.

### 2.8.1 Princípios da Modelagem Assistida por IA

Antes de interagir com qualquer ferramenta de IA, o engenheiro de software deve ter em mente três diretrizes fundamentais:

- **A IA é um Copiloto, Não o Piloto:** A IA é excelente para propor ideias iniciais e checklists, mas carece de contexto do mundo real e senso crítico de negócio. A decisão final de design é sempre do modelador humano.
- **Evite a Alucinação:** Modelos de IA tendem a introduzir detalhes técnicos de baixo nível (como tabelas hash, conexões ou chaves estrangeiras) quando solicitados a modelar um domínio de negócio. Force a IA a manter-se no nível conceitual e de análise.
- **Alinhamento com a Linguagem Ubíqua:** Certifique-se de que os nomes de classes sugeridos pela IA respeitem rigorosamente o glossário e a linguagem compartilhada do negócio definida na Etapa 1.

### 2.8.2 Templates de Prompts Estruturados

Para obter os melhores resultados, utilize a técnica de *Role-Play* (Atribuição de Papel) e forneça restrições claras de design nos seus comandos. Abaixo estão os três prompts recomendados para a Unidade 2.

#### PROMPT 1: Descoberta de Classes via Lista de Categorias de Larman

*Copie e cole o texto abaixo na ferramenta de IA, substituindo as informações entre colchetes pelo seu cenário:*

Aja como um experiente Engenheiro de Software e Analista de Sistemas.

Estamos modelando o domínio de um sistema para:

[Descreva o negócio brevemente, ex: uma Clínica de Saúde e Bem-estar que realiza massoterapia e nutrição].

Com base no método de Craig Larman de "Lista de Categorias de Classes



Conceituais", analise o domínio descrito e identifique possíveis classes candidatas dividindo-as estritamente nas seguintes categorias:

1. Transações de Negócio  
(Eventos vitais que registram atividades no tempo)
2. Papéis de Pessoas ou Organizações envolvidas
3. Lugares ou Estruturas Físicas necessárias para a operação
4. Descrições de Coisas/Serviços  
(Gabaritos que guardam informações estáticas)
5. Outros Sistemas com os quais o nosso sistema colabora

### **PROMPT 2: Análise Linguística de Abbott (Gramatical)**

*Utilize este prompt para extrair classes, atributos e comportamentos de uma descrição de Caso de Uso ou Narrativa de Processo:*

Aja como um Analista de Sistemas especializado no paradigma Orientado a Objetos.

Vou fornecer a você uma narrativa de processo de negócio e quero que você aplique a técnica de Análise Linguística de Abbott para mapear os componentes de software:

[Cole aqui o texto do seu Caso de Uso ou descrição do processo.

Exemplo:

O recepcionista agenda uma nova Sessão de Atendimento para um Paciente.  
O sistema valida se o Especialista possui disponibilidade de horário na Sala correspondente. Ao finalizar, o sistema calcula o preço da

Sessão aplicando o desconto do convênio.]

Faça o seguinte mapeamento estruturado:

1. Substantivos e Frases Nominais:

Identifique os principais substantivos e classifique-os em "Potenciais Classes Conceituais" ou "Atributos" de uma classe.

2. Verbos e Frases Verbais:

Identifique os verbos de ação que indicam responsabilidades lógicas de "Fazer" e atribua-os como métodos das classes identificadas anteriormente.

Explique a lógica de atribuição para cada método, indicando por que o método X pertence à classe Y e não à classe Z, aplicando o princípio de Information Expert.

### PROMPT 3: Combate ao Modelo Anêmico (Classes Ricas)

*Use este prompt para refinar as classes geradas, injetando comportamento inteligente nelas e evitando classes que são meras tabelas de dados passivas:*

Aja como um Arquiteto de Software revisor.

Analisei as classes sugeridas para o meu domínio, mas notei que elas parecem anêmicas, pois possuem apenas dados e métodos getters e setters triviais.

Eu quero construir um modelo rico com Baixo Hiato de Representação (Low Representational Gap).

Para as seguintes classes:

- [Classe 1, ex: SessaoAtendimento  
(atributos: data, hora, status, valor, desconto)]

- [Classe 2, ex: Prontuario  
(atributos: historico, alergias)]

Sugira para cada classe:

1. Pelo menos duas responsabilidades reais de "Fazer", representadas por comportamentos, métodos de regra de negócio ou validações de consistência interna que deveriam ser encapsuladas dentro da própria classe.
2. Justifique sua escolha demonstrando como isso evita que outras classes externas controlem ou violem os dados privados desta classe.
3. Relacione sua justificativa aos conceitos de Encapsulamento, responsabilidade do objeto e à metáfora da "muralha de métodos".

### 2.8.3 Roteiro Prático de Trabalho para as Equipes

Para realizar esta atividade prática em equipe, sigam os passos abaixo:

1. **Definição do Escopo:** Leiam atentamente os requisitos da Etapa 1.2 da Clínica de Saúde e Bem-estar (Nutrição e Massoterapia).
2. **Interação Inicial:** Utilizem o **Prompt 1** na IA de sua escolha para gerar o primeiro rascunho de classes lúdicas.
3. **Análise Crítica Humana:** Examinem a resposta da IA. Risquem todas as sugestões que fujam do negócio (ex: se a IA sugeriu classes como ConexaoBanco, UserLogin ou ValidadorCPF, eliminem-nas).
4. **Refinamento Linguístico:** Apliquem o **Prompt 2** no cenário de "Agendamento de Sessão" para mapear os verbos de negócio que virarão operações (ex: o cálculo de descontos deve virar um método público `calcularValorFinal()` dentro da classe que possui os dados do preço).
5. **Combate à Anemia:** Usem o **Prompt 3** para certificar-se de que a classe `SessaoAtendimento`

possui inteligência interna para validar choques de horários com outras sessões.

6. **Entrega:** Desenhem no papel (ou ferramenta CASE sugerida na Unidade 3) o diagrama de classes resultante do refinamento, contendo nomes de classes, atributos privados, métodos públicos e as respectivas multiplicidades de associação lógicas.

## 2.9 Exercícios de Fixação



Para realizar em papel e caneta, sem consulta, para treinar para a prova somativa A1/3

Estes exercícios a seguir tem o objetivo de praticar a identificação de classes, associações e multiplicidades. Diferentes exercícios para praticar diferentes conceitos. O ideal é resolver os exercícios na sequência.

### Prática 1: Sistema de Biblioteca

**Foco:** identificação de classes, associações e multiplicidades.

**Contexto:** Uma biblioteca universitária deseja informatizar o controle de seu acervo e dos empréstimos realizados por alunos e professores. O sistema deverá controlar as obras existentes, seus exemplares físicos, os usuários e os empréstimos.

#### Requisitos funcionais

- **RF01 – Manter usuários:** O sistema deve permitir cadastrar, alterar, consultar e inativar usuários. Para cada usuário devem ser registrados nome, CPF, e-mail, telefone e data de nascimento.
- **RF02 – Identificar o tipo de usuário:** O sistema deve identificar se o usuário é aluno ou professor. Para alunos devem ser registrados matrícula e curso. Para professores, matrícula funcional e departamento.
- **RF03 – Manter obra:** O sistema deve permitir cadastrar e consultar obras, registrando título, ISBN, ano de publicação e editora.
- **RF04 – Manter autores:** O sistema deve permitir cadastrar autores, registrando nome e nacionalidade. Uma obra pode possuir vários autores, e um autor pode ser responsável por várias obras.
- **RF05 – Controlar exemplares:** O sistema deve controlar individualmente os exemplares

físicos de cada obra. Para cada exemplar devem ser registrados código, data de aquisição e situação.

- **RF06 – Consultar disponibilidade:** O sistema deve permitir consultar uma obra e verificar quais exemplares estão disponíveis para empréstimo.
- **RF07 – Realizar empréstimo:** O sistema deve permitir registrar um empréstimo para um usuário, armazenando a data do empréstimo e a data prevista para devolução.
- **RF08 – Incluir exemplares no empréstimo:** Um empréstimo poderá envolver um ou mais exemplares. Um exemplar não poderá participar simultaneamente de dois empréstimos ainda não devolvidos.
- **RF09 – Registrar devolução:** O sistema deve permitir registrar individualmente a devolução de cada exemplar, armazenando a data efetiva da devolução.
- **RF10 – Consultar empréstimos:** O sistema deve permitir consultar o histórico de empréstimos de um usuário e identificar aqueles que ainda possuem exemplares pendentes.

**Atividade do aluno:** Identifique as classes conceituais, seus atributos e métodos/responsabilidades. Em seguida, estabeleça os relacionamentos, suas multiplicidades e, quando aplicável, generalização/especialização, agregação ou composição.

## Prática 2: Sistema de Clínica Veterinária

**Foco:** maior número de classes e regras de multiplicidade.

**Contexto:** Uma biblioteca universitária deseja informatizar o controle de seu acervo e dos empréstimos realizados por alunos e professores. O sistema deverá controlar as obras existentes, seus exemplares físicos, os usuários e os empréstimos.

### Requisitos funcionais:

- **RF01 – Manter responsáveis:** O sistema deve permitir cadastrar os responsáveis pelos animais, registrando nome, CPF, telefone, e-mail e endereço.
- **RF02 – Manter animais:** O sistema deve permitir cadastrar animais, registrando nome, espécie, raça, sexo, data de nascimento e peso.
- **RF03 – Vincular animais aos responsáveis:** Cada animal deverá possuir um responsável cadastrado. Um responsável poderá possuir vários animais.
- **RF04 – Manter veterinários:** O sistema deve permitir cadastrar veterinários, registrando

nome, CRMV, especialidade, telefone e e-mail.

- **RF05 – Registrar consultas:** O sistema deve permitir registrar consultas, armazenando data, horário, motivo e observações.
- **RF06 – Identificar animal atendido:** Toda consulta deverá estar relacionada a um único animal. Um animal poderá realizar diversas consultas ao longo do tempo.
- **RF07 – Identificar veterinário:** Cada consulta deverá ser realizada por um veterinário. Um veterinário poderá realizar diversas consultas.
- **RF08 – Registrar diagnósticos:** Durante uma consulta, o veterinário poderá registrar nenhum, um ou vários diagnósticos. Para cada diagnóstico devem ser registrados descrição e código CID veterinário, quando existente.
- **RF09 – Manter procedimentos:** O sistema deve permitir cadastrar procedimentos oferecidos pela clínica, contendo código, nome, descrição e valor padrão.
- **RF10 – Registrar procedimentos realizados:** Durante uma consulta poderão ser realizados vários procedimentos. Para cada procedimento realizado devem ser registrados quantidade e valor efetivamente cobrado.
- **RF11 – Calcular valor da consulta:** O sistema deve calcular o valor total da consulta considerando os procedimentos realizados.
- **RF12 – Consultar histórico do animal:** O sistema deve apresentar o histórico de consultas de um animal, incluindo veterinário, diagnósticos e procedimentos realizados.

### Prática 3: Sistema de Locadora de Veículos

**Foco:** generalização/especialização.

**Contexto:** Uma locadora trabalha com diferentes tipos de veículos e deseja controlar sua frota, clientes, reservas e locações.

**Requisitos funcionais:**

- **RF01 – Manter clientes:** O sistema deve cadastrar clientes, registrando CPF, nome, CNH, telefone, e-mail e endereço.
- **RF02 – Manter veículos:** O sistema deve cadastrar os veículos da frota, registrando placa, marca, modelo, ano, quilometragem e valor da diária.
- **RF03 – Classificar veículos:** A frota poderá conter carros, motocicletas e utilitários.

- **RF04 – Registrar informações específicas de carros:** Para carros devem ser registrados quantidade de portas e capacidade do porta-malas.
- **RF05 – Registrar informações específicas de motocicletas:** Para motocicletas devem ser registradas cilindrada e categoria.
- **RF06 – Registrar informações específicas de utilitários:** Para utilitários devem ser registrados capacidade de carga e volume de carga.
- **RF07 – Realizar reserva:** O cliente poderá reservar um veículo informando data prevista de retirada e data prevista de devolução.
- **RF08 – Controlar reservas:** Um cliente poderá possuir várias reservas. Cada reserva deverá estar relacionada a um único veículo.
- **RF09 – Registrar locação:** O sistema deve registrar a retirada do veículo, armazenando data e hora da retirada e quilometragem inicial.
- **RF10 – Registrar devolução:** Na devolução devem ser registrados data e hora, quilometragem final e observações sobre o estado do veículo.
- **RF11 – Calcular valor da locação:** O sistema deverá calcular o valor da locação considerando o período de utilização e o valor da diária do veículo.
- **RF12 – Consultar histórico:** O sistema deverá permitir consultar todas as locações realizadas por determinado cliente e o histórico de utilização de determinado veículo.

#### Prática 4: Sistema de Gestão de Eventos

**Foco:** problema mais completo, associações muitos objetos de uma classe com muito objetivos de outra classe e decisões de modelagem menos explícitas.

**Contexto:** Uma empresa organiza eventos profissionais, como congressos, workshops e seminários. Ela deseja controlar eventos, atividades, palestrantes, participantes e inscrições.

##### Requisitos funcionais:

- **RF01 – Manter eventos:** O sistema deve cadastrar eventos, registrando nome, descrição, data de início, data de término e local.
- **RF02 – Manter atividades:** Um evento poderá oferecer diversas atividades, como palestras, minicursos e mesas-redondas. Para cada atividade devem ser registrados título, descrição, data, horário inicial, horário final e capacidade máxima.

- **RF03 – Manter palestrantes:** O sistema deve cadastrar palestrantes, registrando nome, CPF, e-mail, instituição e minicurrículo.
- **RF04 – Vincular palestrantes às atividades:** Uma atividade poderá possuir um ou mais palestrantes. Um palestrante poderá participar de diferentes atividades e eventos.
- **RF05 – Manter participantes:** O sistema deve cadastrar participantes, registrando nome, CPF, e-mail e telefone.
- **RF06 – Realizar inscrição no evento:** O participante poderá se inscrever em diferentes eventos. Para cada inscrição deverão ser registrados data, situação e valor pago.
- **RF07 – Inscrever-se em atividades:** Depois de inscrito no evento, o participante poderá selecionar as atividades das quais deseja participar.
- **RF08 – Controlar capacidade:** O sistema não deverá permitir novas inscrições em uma atividade quando sua capacidade máxima for atingida.
- **RF09 – Registrar presença:** Para cada atividade selecionada pelo participante, o sistema deverá permitir registrar se ele esteve presente.
- **RF10 – Cancelar participação em atividade:** O participante poderá cancelar sua participação em uma atividade antes de sua realização.
- **RF11 – Consultar programação:** O sistema deverá apresentar a programação completa de um evento com suas atividades, horários e palestrantes.
- **RF12 – Emitir certificado:** O sistema deverá permitir emitir certificado para o participante de acordo com as atividades nas quais sua presença tenha sido registrada.

Os exercícios a seguir fazem parte do conjunto de estudos gerais sobre a Unidade 2. Reforço que se você fizer os exercícios e validar as respostas, estará preparado para acertar todas as questões da prova.

**Conceitual 2.1:** Diferencie uma classe de um objeto (instância) no paradigma orientado a objetos. Explique também como a UML diferencia graficamente a representação de uma classe da representação de um objeto e qual é o significado da identidade de um objeto.

**Conceitual 2.2:** Explique as responsabilidades de “conhecer” (*knowing*) e “fazer” (*doing*) propostas por Craig Larman. Apresente um exemplo de cada tipo de responsabilidade utilizando uma classe de um sistema de clínica.

**Abstração 2.3:** A UML possui diferentes tipos de diagramas, classificados em Diagramas Estruturais e Diagramas Comportamentais. Explique a diferença entre essas duas categorias e indique três exemplos de



diagramas pertencentes a cada categoria.

**Abstração 2.4:** Explique o princípio do encapsulamento utilizando a metáfora da “muralha” de operações públicas. Relacione encapsulamento com atributos privados, passagem de mensagens, coesão e acoplamento.

**Comparativo 2.5:** Compare as perspectivas Conceitual (Modelo de Domínio) e de Software (Design Class Diagram – DCD) de um Diagrama de Classes. Explique o que cada perspectiva procura representar e quais informações aparecem em cada uma delas.

**Comparativo 2.6:** Diferencie associação, agregação, composição e generalização/herança em um Diagrama de Classes. Para cada relacionamento, explique seu significado conceitual e como ele é representado graficamente na UML.

**Cenário 2.7:** Considere o seguinte cenário:

*“Um paciente agenda uma sessão com um especialista. Cada sessão ocorre em uma sala de atendimento. Durante a sessão, o especialista registra informações no prontuário do paciente e, ao final, o sistema registra o pagamento.”*

Utilizando os métodos apresentados por Larman:

- a) identifique pelo menos cinco classes conceituais candidatas;
- b) indique quais foram identificadas pela Lista de Categorias;
- c) aplique a Análise Linguística, identificando substantivos que sugerem classes ou atributos e verbos que sugerem operações.

**Cenário 2.8:** Considere um sistema acadêmico com as classes Aluno, Curso, Disciplina e Professor. Modele conceitualmente os relacionamentos entre essas classes e indique uma multiplicidade adequada em cada extremidade. Em seguida, explique em linguagem natural o significado de cada multiplicidade utilizada.

**Reflexão 2.9:** Explique o conceito de “Hiato de Representação” (Representational Gap). Por que utilizar no modelo os mesmos conceitos e termos empregados pelos especialistas do domínio contribui para a compreensão, manutenção e evolução do software?

**Reflexão 2.10:** Uma classe possui apenas atributos privados e métodos get e set, enquanto todas as regras de negócio relacionadas a ela estão concentradas em uma classe controladora externa. Analise essa solução considerando os conceitos de classe anêmica, responsabilidades, encapsulamento, coesão e acoplamento. Explique como o modelo poderia ser melhorado.

**Questão ENADE 2.1:**

Em um sistema de clínica, diferentes tipos de atendimento -

- como `SessaoNutricional` e `SessaoMassoterapia` – possuem formas próprias de calcular o valor do atendimento. O modelo define uma classe mais geral `Sessao` e permite que cada especialização responda de maneira própria à operação `calcularValor()`.

Considerando os princípios da Orientação a Objetos, avalie as afirmações:

- I. O polimorfismo permite que objetos de subclasses diferentes respondam à mesma mensagem utilizando comportamentos específicos.
- II. Uma relação de herança conceitualmente válida deve atender à Regra do “É Um”, de forma que uma instância da subclasse também possa ser considerada uma instância da superclasse.
- III. A Regra dos 100% estabelece que uma subclasse pode ignorar atributos e associações da superclasse quando eles não forem convenientes à implementação.

É correto o que se afirma em:

- A) I, apenas.
- B) III, apenas.
- C) I e II, apenas.
- D) II e III, apenas.
- E) I, II e III.

**Questão ENADE 2.2:**

Durante a elaboração do modelo de domínio de um sistema de vendas, um analista identificou as classes `Cliente`, `Pedido`, `ItemPedido` e `Produto`. No modelo, um `Cliente` pode realizar vários `Pedidos`; cada `Pedido` deve pertencer a exatamente um `Cliente`; um `Pedido` é formado por um ou mais `ItemPedido`.

Com relação à representação desse domínio em um Diagrama de Classes UML, avalie:

- I. A multiplicidade `0..*` indica que uma instância pode estar relacionada a nenhuma, uma ou várias instâncias da outra classe.
- II. Se `ItemPedido` não possuir existência conceitual independente de `Pedido`, a composição pode representar adequadamente essa relação todo-parte.
- III. A multiplicidade representa o número de atributos que uma classe pode possuir no modelo.

É correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.



# Unidade 3 - Padrões e Uso dos Diagramas

## 3. Padrões e Uso dos Diagramas

### Conteúdo da Unidade

Em construção...

# IV

## Unidade 4 - Guia Prático: Engenharia de Prompts

## 4. Guia Prático: Engenharia de Prompts

### Conteúdo da Unidade

|       |                                             |    |
|-------|---------------------------------------------|----|
| 4.1   | Prompts para Descoberta de Classes          | 59 |
| 4.1.1 | Princípios da Modelagem Assistida por IA    | 59 |
| 4.1.2 | Templates de Prompts Estruturados           | 60 |
| 4.1.3 | Roteiro Prático de Trabalho para as Equipes | 62 |

### 4.1 Prompts para Descoberta de Classes

Este guia prático foi desenvolvido para orientar os acadêmicos no uso crítico e responsável de ferramentas de Inteligência Artificial Generativa (como Gemini, ChatGPT ou Claude) durante o processo de modelagem de sistemas. O objetivo é utilizar a IA como um copiloto para acelerar a descoberta de classes conceituais, atributos e responsabilidades, aplicando os métodos clássicos de Craig Larman e a análise gramatical de Abbott.

---

#### 4.1.1 Princípios da Modelagem Assistida por IA

Antes de interagir com qualquer ferramenta de IA, o engenheiro de software deve ter em mente três diretrizes fundamentais:

- **A IA é um Copiloto, Não o Piloto:** A IA é excelente para propor ideias iniciais e checklists, mas carece de contexto do mundo real e senso crítico de negócio. A decisão final de design é sempre do

modelador humano.

- **Evite a Alucinação:** Modelos de IA tendem a introduzir detalhes técnicos de baixo nível (como tabelas hash, conexões ou chaves estrangeiras) quando solicitados a modelar um domínio de negócio. Force a IA a manter-se no nível conceitual e de análise.
- **Alinhamento com a Linguagem Ubíqua:** Certifique-se de que os nomes de classes sugeridos pela IA respeitem rigorosamente o glossário e a linguagem compartilhada do negócio definida na Etapa 1.

#### 4.1.2 Templates de Prompts Estruturados

Para obter os melhores resultados, utilize a técnica de *Role-Play* (Atribuição de Papel) e forneça restrições claras de design nos seus comandos. Abaixo estão os três prompts recomendados para a Unidade 2.

##### **PROMPT 1: Descoberta de Classes via Lista de Categorias de Larman**

*Copie e cole o texto abaixo na ferramenta de IA, substituindo as informações entre colchetes pelo seu cenário:*

Aja como um experiente Engenheiro de Software e Analista de Sistemas. Estamos modelando o domínio de um sistema para: [Descreva o negócio brevemente, ex: uma Clínica de Saúde e Bem-estar que realiza massoterapia e nutrição].

Com base no método de Craig Larman de "Lista de Categorias de Classes Conceituais", analise o domínio descrito e identifique possíveis classes candidatas dividindo-as estritamente nas seguintes categorias: 1. Transações de Negócio (Eventos vitais que registram atividades no tempo) 2. Papéis de Pessoas ou Organizações envolvidas 3. Lugares ou Estruturas Físicas necessárias para a operação 4. Descrições de Coisas/Serviços (Gabaritos que guardam informações estáticas) 5. Outros Sistemas com os quais o nosso sistema colabora

Restrições Mandatórias: - Forneça apenas classes lógicas de negócio. Não inclua nenhuma classe de infraestrutura técnica (ex: Banco de Dados, Controller, Tela, API, Login). - Justifique brevemente por que cada classe foi sugerida. - Utilize estritamente os termos da seguinte linguagem ubíqua: [Insira os termos chave do seu glossário, ex: Paciente, Especialista, Sessao].



**PROMPT 2: Análise Linguística de Abbott (Gramatical)**

*Utilize este prompt para extrair classes, atributos e comportamentos de uma descrição de Caso de Uso ou Narrativa de Processo:*

Aja como um Analista de Sistemas especializado no paradigma Orientado a Objetos. Vou fornecer a você uma narrativa de processo de negócio e quero que você aplique a técnica de Análise Linguística de Abbott para mapear os componentes de software:

[Cole aqui o texto do seu Caso de Uso ou descrição do processo, ex: O recepcionista agenda uma nova Sessão de Atendimento para um Paciente. O sistema valida se o Especialista possui disponibilidade de horário na Sala correspondente. Ao finalizar, o sistema calcula o preço da Sessão aplicando o desconto do convênio.]

Faça o seguinte mapeamento estruturado: 1. Substantivos e Frases Nominais: Identifique os principais substantivos e classifique-os em "Potenciais Classes Conceituais" ou "Atributos" de uma classe. 2. Verbos e Frases Verbais: Identifique os verbos de ação que indicam responsabilidades lógicas ("Fazer") e atribua-os como métodos das classes identificadas anteriormente.

Explique a sua lógica de atribuição para cada método (por que o método X pertence à classe Y e não à classe Z, aplicando o princípio de Information Expert).

**PROMPT 3: Combate ao Modelo Anêmico (Classes Ricas)**

*Use este prompt para refinar as classes geradas, injetando comportamento inteligente nelas e evitando classes que são meras tabelas de dados passivas:*

Aja como um Arquiteto de Software revisor. Analisei as classes sugeridas para o meu domínio, mas notei que elas parecem anêmicas (possuem apenas dados e métodos getters e setters triviais). Eu quero construir um modelo rico com Baixo Hiato de Representação (Low Representational Gap).

Para as seguintes classes: - [Classe 1, ex: SessaoAtendimento (atributos: data, hora, status, valor, desconto)] - [Classe 2, ex: Prontuario (atributos: historico, alergias)]

Sugira para cada classe: 1. Pelo menos duas responsabilidades reais de "Fazer" (comportamentos, métodos de regra de negócio ou validações de consistência interna) que deveriam ser encapsuladas dentro da própria classe para protegê-la. 2. Justifique sua escolha demonstrando como isso evita que outras classes externas controlem e violem os dados privados desta classe (respeitando o

Encapsulamento e a metáfora da "muralha de métodos").

### 4.1.3 Roteiro Prático de Trabalho para as Equipes

Para realizar esta atividade prática em equipe, sigam os passos abaixo:

1. **Definição do Escopo:** Leiam atentamente os requisitos da Etapa 1.2 da Clínica de Saúde e Bem-estar (Nutrição e Massoterapia).
2. **Interação Inicial:** Utilizem o **Prompt 1** na IA de sua escolha para gerar o primeiro rascunho de classes lúdicas.
3. **Análise Crítica Humana:** Examinem a resposta da IA. Risquem todas as sugestões que fujam do negócio (ex: se a IA sugeriu classes como `ConexaoBanco`, `UserLogin` ou `ValidadorCPF`, eliminem-nas).
4. **Refinamento Linguístico:** Apliquem o **Prompt 2** no cenário de "Agendamento de Sessão" para mapear os verbos de negócio que virarão operações (ex: o cálculo de descontos deve virar um método público `calcularValorFinal()` dentro da classe que possui os dados do preço).
5. **Combate à Anemia:** Usem o **Prompt 3** para certificar-se de que a classe `SessaoAtendimento` possui inteligência interna para validar choques de horários com outras sessões.
6. **Entrega:** Desenhem no papel (ou ferramenta CASE sugerida na Unidade 3) o diagrama de classes resultante do refinamento, contendo nomes de classes, atributos privados, métodos públicos e as respectivas multiplicidades de associação lógicas.